



Degree Project in Computer Science and Engineering

Second cycle, 30 credits

An RTOS for the Limited-Preemptive Three-Phase Execution Model

Implementation and Evaluation on a COTS Multi-Core Platform

YI REN

An RTOS for the Limited-Preemptive Three-Phase Execution Model

Implementation and Evaluation on a COTS Multi-Core Platform

YI REN

Master's Programme, Embedded Systems, 120 credits

Date: June 8, 2026

Supervisor: Matthias Becker

Examiner: Ingo Sander

School of Electrical Engineering and Computer Science

Swedish title: Ett RTOS för den begränsat preemptiva trefasiga
exekveringsmodellen

Swedish subtitle: Implementering och utvärdering på en COTS-baserad
flerkärnig plattform

Abstract

This thesis presents the implementation and evaluation of a Real-Time Operating System (RTOS) that supports the limited-preemptive three-phase execution model on a Commercial Off-The-Shelf (COTS) multi-core platform. The phased execution model is proposed to improve predictability on COTS platforms by structuring task execution into a read phase, an execution phase, and a write phase, thereby reducing contention for shared resources. Previous work has shown the feasibility of introducing limited preemption into the phased execution model. However, the approaches are mainly based on theoretical analysis and simulations, while practical implementation and evaluations remain limited.

To address this problem, this thesis implements an RTOS kernel that conforms to the theoretical model of the limited-preemptive three-phase execution model on a dual-core Arm Cortex-M7 platform. The implementation uses partitioned fixed-priority scheduling, the Overhead Minimizing Preemption Method (OMPM) for realizing preemptions in execution phases, and preemption thresholds to limit preemptions. An arbitration mechanism based on priority is also implemented in this thesis for controlling shared memory access between the two cores.

This thesis provides a practical implementation approach that enables the theoretical model to execute on an actual COTS multi-core platform. Experimental results show that the scheduling behaviors largely conform to the theoretical model, demonstrating the practical feasibility of the approach. Runtime overheads introduced by the limited-preemptive approach are also experimentally evaluated in this thesis, providing practical references for calibrating overhead-related parameters and assumptions in the theoretical model.

Keywords

Real-Time Operating Systems, Fixed-Priority Scheduling, Phased Execution Models, Limited Preemption, Multi-Core Systems

Sammanfattning

Denna avhandling presenterar implementationen och utvärderingen av ett realtidsoperativsystem (RTOS) som stöder den begränsat preemptiva trefasiga exekveringsmodellen på en flerkärnig Commercial Off-The-Shelf-plattform (COTS). Den fasindelade exekveringsmodellen föreslås för att förbättra förutsägbarheten på COTS-plattformar genom att strukturera exekveringen av uppgifter i en läsfas, en exekveringsfas och en skrivfas, vilket därmed minskar konkurrens om delade resurser. Tidigare arbete har visat genomförbarheten av att introducera begränsad preemption i den fasindelade exekveringsmodellen. Dessa metoder är dock huvudsakligen baserade på teoretisk analys och simuleringar, medan praktiska implementationer och utvärderingar fortfarande är begränsade.

För att adressera detta problem implementerar denna avhandling en RTOS-kärna som överensstämmer med den teoretiska modellen för den begränsat preemptiva trefasiga exekveringsmodellen på en dubbelkärnig Arm Cortex-M7-plattform. Implementationen använder partitionerad fastprioritetsschemaläggning, Overhead Minimizing Preemption Method (OMPM) för att realisera preemptioner under exekveringsfaser samt preemptionströsklar för att begränsa preemptioner. En prioritetbaserad arbitreringsmekanism implementeras också i denna avhandling för att kontrollera åtkomst till delat minne mellan de två kärnorna.

Denna avhandling tillhandahåller en praktisk implementationsmetod som möjliggör exekvering av den teoretiska modellen på en faktisk flerkärnig COTS-plattform. Experimentella resultat visar att schemaläggningsbeteendena i stor utsträckning överensstämmer med den teoretiska modellen, vilket demonstrerar metodens praktiska genomförbarhet. Runtime-overheads som introduceras av den begränsat preemptiva metoden utvärderas också experimentellt i denna avhandling och tillhandahåller praktiska referenser för kalibrering av overheadrelaterade parametrar och antaganden i den teoretiska modellen.

Nyckelord

Realtidsoperativsystem, Fastprioritetsschemaläggning, Flerfasiga exekveringsmodeller, Begränsad preemption, Flerkärniga system

Acknowledgments

I would like to express my sincere gratitude to my supervisor, Matthias Becker, for his continuous support and valuable guidance throughout this thesis.

I would also like to thank my examiner, Ingo Sander, for his support and helpful guidance during the project.

Generative AI tools were used for grammar checking and translating the abstract into Swedish. The full thesis was written by the author, and all technical content, including the implementation and evaluation, was developed by the author as well.

Stockholm, June 2026

Yi Ren

Contents

1	Introduction	1
1.1	Background	1
1.2	Problem	2
1.2.1	Original problem and definition	2
1.2.2	Scientific and engineering issues	2
1.3	Purpose	3
1.4	Ethics and sustainability	3
1.5	Goals	3
1.6	Research methodology	4
1.7	Delimitations	4
1.8	Structure of the thesis	5
2	Background	7
2.1	Phased execution models	7
2.1.1	Motivation and concept of phased execution	7
2.1.2	Predictable Execution Model (PREM) model	8
2.1.3	Acquisition-Execution-Restitution (AER) model	8
2.2	Preemption in execution phases	9
2.2.1	Waiting Time Minimizing Preemption Method (WMPM)	9
2.2.2	Overhead Minimizing Preemption Method (OMPM)	10
2.3	Limited preemption in scheduling	10
2.3.1	Preemption thresholds	10
2.3.2	Non-preemptive sections	11
2.4	Related work	11
2.4.1	Phased execution models	12
2.4.2	Limited preemption	12
2.5	Summary	13

3	System model	15
3.1	Platform model	15
3.2	Application model	16
3.3	Execution model	17
3.3.1	Scheduling policy	17
3.3.2	Preemption model	17
3.3.3	Limited preemption	17
4	Methodology	19
4.1	Research process	19
4.2	Engineering methodology	20
4.2.1	Hardware platform to be used	20
4.2.2	RTOS kernel design	20
4.2.3	Overhead evaluation	21
4.3	Evaluation framework	21
4.4	Assessing reliability and validity of the data collected	22
4.4.1	Validity	22
4.4.2	Reliability	22
5	Implementation	25
5.1	Implementation on a single-core side	25
5.1.1	Concurrent task execution	25
5.1.1.1	Context switch	25
5.1.1.2	Blocking mechanism	27
5.1.2	Memory management	29
5.1.2.1	Memory layout configuration	29
5.1.2.2	Management of core-local and shared mem- ory	31
5.1.3	Scheduler design	35
5.1.3.1	Stack invariant under OMPM	35
5.1.3.2	Preemption	36
5.1.3.3	CPU yield	38
5.2	Migration to the dual-core environment	39
5.2.1	Synchronization	40
5.2.2	Arbiter design	41
6	Evaluation	45
6.1	Scheduling behaviors	45
6.1.1	Logging mechanism	45
6.1.2	Scheduling analysis	46

6.1.2.1	Comparison with the theoretical scheduling example	46
6.1.2.2	Fully-preemptive mode in execution phases	48
6.1.2.3	Limited-preemptive mode with preemption thresholds	49
6.1.2.4	Preemption with insufficient core-local memory	49
6.1.2.5	Fully non-preemptive mode	50
6.1.2.6	Dual-core scheduling behavior	51
6.2	Overhead evaluation	53
6.2.1	Overhead evaluation of task switching	54
6.2.2	Overhead evaluation of inter-core synchronization	56
7	Conclusions and future work	59
7.1	Conclusions	59
7.2	Limitations	60
7.2.1	Static assignments of task code	60
7.2.2	Static assignments of task stacks	60
7.2.3	Task data isolation	60
7.2.4	Timing-related limitations	61
7.3	Future work	61
7.3.1	Closer alignment with the theoretical model	61
7.3.2	Maintain an explicit preemption stack	62
	References	63
A	Complete source code repository	67

List of Figures

3.1	Simplified block diagram of the platform model	15
5.1	Example of task sets using bitmaps	28
5.2	Memory layout of SRAM	31
5.3	Example of allocation in the memory pool	33
5.4	Example of managing core-local memory	34
5.5	Example of arbitration failure when not well synchronized	40
6.1	Log format	46
6.2	Theoretical scheduling behavior of the example task set	47
6.3	Actual scheduling behavior of the example task set	47
6.4	Scheduling behavior in fully-preemptive mode	48
6.5	Scheduling behavior in limited-preemptive mode	49
6.6	Scheduling behavior when having insufficient memory	50
6.7	Scheduling behavior in fully non-preemptive mode	51
6.8	Scheduling behavior in the dual-core environment	51
6.9	Box plot of task switching times	55
6.10	Systicks on two cores without synchronization	57
6.11	Systicks on two cores after synchronization	57
6.12	Measurement of timing difference	57

List of Tables

6.1	Evaluation result of task switching times (clock cycles)	54
-----	--	----

Listings

5.1	TCB data structure	26
5.2	Key logic of context switch	27
5.3	Key parts of the linker script	30
5.4	Place main stack into the data Tightly Coupled Memory (TCM)	30
5.5	Pseudocode of a task	32
5.6	Pseudocode of the preemption path	37
5.7	Pseudocode of the function finding the highest-prioritized available task	38
5.8	Pseudocode of the yield path	39
5.9	Pseudocode of the function finding the top task in the preemption stack	39
5.10	Pseudocode of synchronization	41
5.11	Arbiter data structure	42
5.12	Pseudocode of arbiter	43

List of acronyms and abbreviations

AER	Acquisition-Execution-Restitution
API	Application Programming Interface
COTS	Commercial Off-The-Shelf
FPSCR	Floating-Point Status and Control Register
FPU	Floating Point Unit
GPIO	General-Purpose Input/Output
IDE	Integrated Development Environment
LIFO	Last-in, First-Out
MSP	Main Stack Pointer
OMPM	Overhead Minimizing Preemption Method
PIC	Position-Independent Code
PREM	Predictable Execution Model
PSP	Process Stack Pointer
RTOS	Real-Time Operating System
SDK	Software Development Kit
SP	Stack Pointer
TCB	Task Control Block
TCM	Tightly Coupled Memory
WCET	Worst-Case Execution Time
WMPM	Waiting Time Minimizing Preemption Method

Chapter 1

Introduction

In this thesis, we investigate the practical realization of the phased execution model with limited preemption in execution phases. Specifically, we focus on the implementation of a **Real-Time Operating System (RTOS)** kernel which conforms to the theoretical model, and on the experimental evaluation of the resulting runtime overheads on a **Commercial Off-The-Shelf (COTS)** multi-core platform.

1.1 Background

Modern real-time embedded systems are increasingly being built on **COTS** platforms, as the performance of **COTS** platforms can be several orders of magnitude faster than custom-made ones [1]. However, these **COTS** platforms are primarily designed to increase the average-case performance, and rarely care about the real-time constraints [2]. The contention for memory and other shared resources by multiple active components, such as CPU cores, can lead to timing degradation and affect the **Worst-Case Execution Time (WCET)** analysis [3].

In order to address this challenge, the phased execution model is proposed [3], which enforces a coarse-grained scheduling of shared resources on the software level. Take the contention of shared memory of CPU cores as an example. The task execution is divided into memory phases and execution phases, where access to shared memory is only allowed in memory phases, and only resources exclusive to the core (e.g., scratchpad memory or level-1 cache) can be accessed in execution phases. Only one memory phase can be active at a time to avoid interference between different CPU cores. Moreover, the memory phases can be further refined into read and write phases, leading to the

three-phase execution model, where execution is divided into read, execute, and write phases.

Phased execution models are commonly used in non-preemptive ways [3, 4], but the inflexibility of non-preemptive scheduling may cause the deadline to be missed [5]. Recent work explores the potential of limited-preemptive execution phases [6, 7]. Methods such as the **Waiting Time Minimizing Preemption Method (WMPM)** and the **Overhead Minimizing Preemption Method (OMPM)** are introduced to allow preemptions in execution phases [6]. Limited preemption strategies, such as preemption thresholds and non-preemptive sections, are also proposed to balance runtime overheads [7]. By introducing limited preemption in execution phases, it becomes possible to reduce the blocking time while preserving predictability.

1.2 Problem

Introducing limited preemption in the execution phase has been shown to have great potential in research [6]. However, the evaluation of the theoretical model was conducted with simulation environments, and has not been implemented on actual **COTS** multi-core platforms. We therefore ask:

*How can the limited-preemptive three-phase execution model be implemented on a **COTS** multi-core platform, and how large are the runtime overheads it introduces compared to a fully non-preemptive baseline?*

1.2.1 Original problem and definition

The original problem addressed in this thesis is the practical realization of the limited-preemptive three-phase execution model on an actual **COTS** multi-core platform, together with the quantitative evaluation of the overheads of introducing limited preemption into the three-phase execution model.

1.2.2 Scientific and engineering issues

From a scientific aspect, the approach of introducing limited preemption to the phased execution model has been proven to be feasible and has the potential to improve schedulability compared with non-preemptive phased execution models, with the overheads of the **RTOS** kernel not evaluated on actual **COTS** platforms, making it lack persuasiveness.

From an engineering aspect, the theory of limited preemption of phased execution models has not been implemented on real **COTS** multi-core

platforms, and it remains unclear whether and how the approach can be realized in practice.

1.3 Purpose

The purpose of this thesis is to implement the theoretical model of the three-phase execution model with the approach of limited preemption in execution phases on an actual **COTS** multi-core platform, and experimentally evaluate the overheads of the approach. This thesis contributes to the proof of the theoretical model in practice, provides an actual and precise evaluation of the overheads, and helps to calibrate the parameters regarding overheads in the theory.

This thesis also provides a practical way of realizing the theory of the limited-preemptive three-phase execution model, which has great potential for enhancing the performance of executions on **COTS** multi-core platforms without sacrificing the predictability of real-time applications.

1.4 Ethics and sustainability

This thesis does not involve human subjects, personal data, or ethically sensitive information, so no direct ethical risks are anticipated. From a sustainability perspective, this thesis may contribute to safer and more efficient design in embedded systems, which might potentially reduce consumption of computational resources.

1.5 Goals

The goal of this thesis is to realize the theoretical model of the limited-preemptive three-phase execution model and evaluate the runtime overheads. This has been divided into the following two sub-goals:

1. Implement an **RTOS** kernel that conforms to the theoretical model of the three-phase execution model with the approach of limited preemption in execution phases on an actual **COTS** multi-core platform.
2. Experimentally evaluate the runtime overheads of introducing the preemption approach by comparing the performance of the kernel in limited-preemptive and non-preemptive modes.

1.6 Research methodology

This thesis employs an engineering-oriented research methodology, which implements a designated **RTOS** kernel and does experimental evaluations on a real **COTS** hardware.

The implementation of the designated **RTOS** kernel is carried out step by step. As the first step, a basic fixed-priority scheduler for three-phase tasks on a single core is implemented. In this step, the context switching of tasks and the management of core-local memory are implemented. Next, an **RTOS** kernel applicable to a dual-core platform is implemented on the basis of the basic scheduler. The kernel at this step will be fully non-preemptive and will serve as the baseline of future evaluations. Afterwards, preemption in execution phases is introduced to the **RTOS** kernel, following the theoretical model of limited-preemptive three-phase tasks. The **RTOS** kernel is able to operate in either fully non-preemptive mode or limited-preemptive mode, decided by a config file.

For the evaluation of the approach, selected benchmark applications that are transformed for the three-phase execution model, as well as synthetically generated applications, are used. The runtime overheads of introducing preemption in execution phases are quantitatively measured by comparing the performance of the **RTOS** kernel in both modes.

Simulation approaches are not used in this thesis, as we focus on the actual implementation of the theoretical model and intend to evaluate the actual runtime overheads.

1.7 Delimitations

Due to the restrictions of practice and workload, this thesis has several limitations as follows.

1. Only the overheads of introducing the preemption approach are evaluated in this thesis. The evaluation of schedulability is not directly part of the thesis, as it involves formal timing analysis of the scheduling algorithm and exceeds the reasonable workload of a master's thesis.
2. The comparison of possible variants to realize preemption in phased execution models is also not considered in this project. This project will focus on **OMPM** since this variant has been shown to work better.

3. A partitioned fixed-priority scheduler will be implemented in this thesis, and preemption thresholds will be used. Methods other than preemption thresholds for limiting preemptions will not be implemented.

1.8 Structure of the thesis

This thesis is structured as follows:

- Chapter 2 presents relevant background information on limited-preemptive three-phase execution models.
- Chapter 3 introduces the formal system model used in this thesis.
- Chapter 4 describes the methodology used to address the problem.
- Chapter 5 presents the implementation of the RTOS kernel in detail.
- Chapter 6 evaluates the implemented RTOS kernel.
- Chapter 7 summarizes the thesis and discusses limitations and future work.

Chapter 2

Background

This chapter provides basic background information about phased execution models and limited preemption used for running real-time applications on a **COTS** multi-core platform. Additionally, related work about the limited-preemptive phased execution model will be introduced in this chapter.

2.1 Phased execution models

The phased execution model is an approach to coarse-scheduling of access to shared resources on the software level, addressing the challenge of contention, which could cause unpredictable timing behaviors.

2.1.1 Motivation and concept of phased execution

One of the greatest challenges for running real-time applications on **COTS** platforms is the contention for shared hardware resources. **COTS** platforms are primarily designed for a higher average performance, and are designed with little or no attention to the time requirements of real-time applications [3]. Shared hardware resources on these platforms are governed by performance-oriented arbiters, and their behaviors are opaque to users [2]. All potential interleavings need to be considered, and this leads to the explosion of the **WCET** analysis complexity [8].

To address this challenge, one feasible way is to coarse-schedule the access to shared resources on the software level. All accesses to shared resources are confined to explicit phases, and interference occurs only at well-defined points. This allows explicit scheduling and analysis of the interference among shared resources, and significantly improves the predictability of task execution.

The phased execution model is such an approach [3]. It represents a class of execution models, where the execution of tasks is structured into phases with distinct resource access characteristics. The execution alternates between phases, and the resource contention can therefore be minimized to a predictable level.

2.1.2 Predictable Execution Model (PREM) model

The **Predictable Execution Model (PREM)** model is a typical model that conforms to the phased execution model [3]. In the **PREM** model, task execution is divided into a sequence of non-preemptive scheduling intervals. These intervals are classified into predictable intervals and compatible intervals. In predictable intervals, all required data are prefetched at the beginning, thus the interval can be executed predictably. In compatible intervals, no special provisions are required before and during execution. Interrupts are not allowed in predictable intervals, but are allowed in compatible intervals with bounded blocking time. Furthermore, preemption by a higher-priority job can only happen at the end of a scheduling interval, which means that both predictable intervals and compatible intervals are non-preemptive.

The **PREM** model highly constrains the execution of tasks and schedules from a coarse-grained level. By introducing the concept of predictable intervals and minimizing the effect of compatible intervals, the execution of tasks in the **PREM** model becomes highly predictable. The execution time of predictable intervals is kept constant, and contentions to share resources are restricted to compatible intervals and can be well-defined, rather than relying on arbiters on **COTS** platforms without timing guarantees.

2.1.3 Acquisition-Execution-Restitution (AER) model

The **Acquisition-Execution-Restitution (AER)** model is another representative of phased execution models [4]. The **AER** model is designed and commonly used for solving memory contention and exploiting parallelism on multi-core environments [9, 10, 4]. As shown in its name, task executions in the **AER** model are divided into three phases, *Acquisition*, *Execution*, and *Restitution*. In the acquisition phase, all necessary data for task execution is loaded into the core-local memory. In the execution phase, the core executes on its local memory without the need to access the shared memory. Finally, in the restitution phase, the results of the computation are written back to the shared

memory.

The **AER** model can be seen as a generalization of the **PREM** model [10]. In particular, the execution phase in the **AER** model corresponds to the predictable interval in the **PREM** model, and the acquisition phase and restitution phase correspond to the compatible interval in the **PREM** model. The **AER** model provides a more explicit structure for the typical dataflow of real-time applications.

2.2 Preemption in execution phases

In phased execution models such as **PREM** or **AER**, the execution of tasks is typically considered non-preemptive [3, 4]. This reduces the influence of other tasks while executing, and makes the execution highly predictable. However, due to the inflexible nature of non-preemptive scheduling, it may be possible for some tasks to miss their deadlines while resources are utilized by other tasks [5].

To address this issue, preemption during the execution phase can be carefully considered. [6] proposed detailed methods and considerations on realizing preemption in phased execution models. When handling preemptions in phased execution models, not only the execution context of the task but also the local copies of the task's data need to be considered. They refer to the data that needs to be managed in case of preemption as *intermediate data*, and proposed two methods for handling it, namely **WMPM** and **OMPM**.

2.2.1 Waiting Time Minimizing Preemption Method (WMPM)

The **WMPM** method aims at minimizing the waiting time of the preempting task. In **WMPM**, preemption is always ensured to take place regardless of the current usage of core-local memory to minimize the waiting time. To achieve this, the intermediate data of the preempted task that is stored in the core-local memory will always be written back to the shared memory, which guarantees sufficient local memory for the execution of the preempting task.

The approach of minimizing the waiting time in the **WMPM** method introduces additional overhead for operations on shared memory. The write-back and read-back of intermediate data of the preempted task introduce two additional memory phases. The **WMPM** method can be seen as sacrificing overhead for responsiveness.

2.2.2 Overhead Minimizing Preemption Method (OMPM)

In contrast with **WMPM**, the **OMPM** method aims to minimize the overheads in preemption. In the **OMPM** method, intermediate data of the preempted task is kept in the core-local memory, and will not be written back to the shared memory to give space to the preempting task. Whether a preemption can take place or not is decided by the usage of the core-local memory. When the free space in the core-local memory is not enough for the intermediate data of the preempting task, preemption will not occur.

The **OMPM** method does not introduce additional overhead in memory operations. The intermediate data will be kept in its place when a task is preempted, and the execution of the preempted task can resume once the execution of the preempting task finishes. And at a price, it is not always guaranteed that a preemption can take place, and it does not provide the highest responsiveness.

2.3 Limited preemption in scheduling

Methods such as **WMPM** and **OMPM** provide the feasibility of realizing preemptions in the execution phase of the phased model. However, while fully-preemptive scheduling provides the highest responsiveness, its overheads may be unacceptable for real-time applications on hardware platforms [5]. As a result, the approach of limited preemption becomes worth considering.

[7] provides several methods for implementing limited preemption on phased execution models. Two representatives, namely *preemption thresholds* and *non-preemptive sections*, are introduced.

2.3.1 Preemption thresholds

The concept of preemption thresholds was proposed in [11], and [7] gives the detailed considerations of preemption thresholds for phased execution models.

In the method of preemption thresholds, each task is assigned a nominal priority and a preemption threshold. The nominal priority of the task is used for scheduling in the ready queue before entering its read phase. When a task is scheduled to run, i.e., enters its read phase, its priority will be increased to its preemption threshold. A task can only be preempted during the execution phase, when the nominal priority of the preempting task is higher than the preemption threshold of the preempted task. Otherwise, the preemption will not take place.

The method of preemption thresholds reduces unnecessary preemptions. Only tasks that are higher than the preemption threshold, instead of the nominal priority, can preempt the running task. When using the **OMPM** method for implementing preemption, i.e., when intermediate data is kept in core, using preemption thresholds can significantly reduce the memory requirement.

2.3.2 Non-preemptive sections

The method of non-preemptive sections is another approach for limited preemption. In this method, a non-preemptive section will be configured at the beginning of the execution phase, prolonging the non-preemptive read phase. In this way, a task can be prevented from being preempted immediately after entering its execution phase, and can consequently reduce the number of tasks that simultaneously occupy the core-local memory and therefore reduce the overhead of preemptions. When the length of the non-preemptive section is set to 0, the execution phase of the task is exactly fully preemptive; and when it is set to the length of the execution phase, the task turns out to be fully non-preemptive.

The approach of non-preemptive sections has a similarity with the deferred preemption model [12]. In the deferred preemption model, a maximum non-preemptive section is defined for each task, and preemption by a higher priority task will be deferred. However, the start and end times of the non-preemptive sections are not fixed and can vary across different executions. In the approach of non-preemptive sections for phased execution models, the non-preemption section starts exactly at the beginning of the execution phase, and remains for a fixed length. This makes the execution of tasks more predictable and the model easier to implement.

2.4 Related work

The field of running real-time applications on **COTS** platforms has been thoroughly researched. Related work has been conducted mainly in the field of the study on phased execution models and how to apply them on **COTS** platforms. Approaches regarding how to introduce preemptions to the phased execution models have also been researched, with methods of implementing limited preemption also considered.

2.4.1 Phased execution models

With the principle of coarse-scheduling shared resources by structuring the execution of tasks, [3] presented the idea of phased execution models, and extended it to the **PREM** model. The **PREM** model was for a general theoretical consideration of shared resources such as I/O peripherals and shared memory. In the later work of [4], the **AER** model was proposed as a generalization of the **PREM** model and was used in a more engineering-oriented context. The **AER** model was first applied for a predictable flight management system on a multicore processor in [4], and was soon generalized in [9], [10], etc., for a more general usage. In the work proposed in [13], they extended the phased execution models to a system-level operating system framework, which systematically integrated scheduling and management of scratchpad memory.

Automated methods for generating tasks compatible with the phased execution models have also been developed. In [14], a complete framework was proposed for automated generation of time-predictable executables on multi-core platforms, which transforms synchronous programs into the **AER**-based task format.

2.4.2 Limited preemption

The approach of introducing preemptions to the phased execution models has also been well-studied. The idea was proposed in [15], where a fully-preemptive scheduling algorithm was considered for multi-core systems, and preemption could take place in both the memory phase and execution phase. Models with preemptions only allowed in execution phases were considered in the analysis of [16] and [17] in a theoretical manner, and timing and schedulability analysis were also given in their studies. [6] proposed a more detailed implementation of preemptions in the execution phases, such as the method of **WMPM** and **OMPM**, with the consideration of restrictions on local memory.

Introducing limited preemption has also been studied in previous works. Several approaches regarding limited-preemptive scheduling for real-time systems, such as preemption thresholds and deferred preemptions, were introduced in [5]. Considerations about introducing these limited preemption methods into the phased execution models were proposed in [7]. They made modifications to the previous works and proposed methods, including preemption thresholds and configuring non-preemptive sections, which are more suitable for phased execution models.

2.5 Summary

This chapter provides an overview of the background of running real-time applications on a **COTS** platform. The concept of using phased execution models for solving predictability issues and several representatives, such as **PREM** and **AER**, is introduced in this chapter. Methods such as **WMPM** and **OMPM**, which are proposed for implementing preemptions in phased execution models, are also introduced. Introduction of the approaches to applying limited preemption in phased execution models is also included, with preemption thresholds and non-preemptive sections as two representatives.

It has been shown that running real-time applications on a **COTS** platform using phased execution models is feasible and has sufficient precedents. Approaches to introducing limited preemption in phased execution models have also been well-studied, and implementation details are also carefully considered in their work. Together, they contribute to a solid background for this thesis.

Chapter 3

System model

This chapter formally defines the system model used in this thesis. The following part of this thesis, including the implementation and evaluation, strictly follows the model defined.

3.1 Platform model

In this thesis, we consider a dual-core platform, which has two identical cores. Each core has its own core-local memory of size S that can be used to store tasks' intermediate data, and the two cores can access the shared memory through a shared bus. Each core has its own instance of the **RTOS** kernel operating independently. Figure 3.1 shows the simplified block diagram of the platform model considered in this thesis.

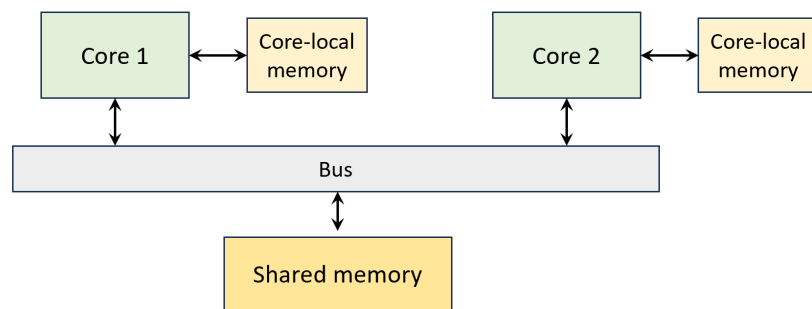


Figure 3.1: Simplified block diagram of the platform model

3.2 Application model

The application consists of a set of independent periodic tasks

$$\tau = \{\tau_1, \tau_2, \dots, \tau_n\}.$$

The tasks are structured according to the **AER** model introduced in Section 2.1.3. That is, each task consists of three sequential phases:

- A *read* phase, where all necessary data and instructions are copied from the shared main memory to the core-local memory.
- An *execution* phase, where the core executes on its local memory and does not access the shared memory.
- A *write* phase, where the results of computation are written back to the shared memory.

Accordingly, each task τ_i is represented by

$$\tau_i = (T_i, D_i, M_i, P_i, O_i, C_i^r, C_i^e, C_i^w),$$

where:

- T_i is the period,
- $D_i \leq T_i$ is the deadline,
- M_i is the memory footprint,
- P_i is the fixed priority,
- O_i is the initial offset,
- C_i^r , C_i^e , and C_i^w are the execution times of the read, execute, and write phases.

The total execution time C_i of task τ_i is the sum of the three phases, i.e.,

$$C_i = C_i^r + C_i^e + C_i^w.$$

The memory footprint M_i of task τ_i is defined as

$$M_i = I_{m_i} + D_{m_i} + S_{m_i},$$

where I_{m_i} is the code size, D_{m_i} is the total data size, and S_{m_i} is the maximum stack usage of the task.

We assume that the memory footprint of every single task does not exceed the size of the core-local memory, i.e., $M_i \leq S$ for all τ_i .

3.3 Execution model

For the execution model of this thesis, we consider the scheduling policy, the preemption mechanism, and the limited preemption method.

3.3.1 Scheduling policy

The partitioned fixed-priority scheduling is used in this thesis. Tasks are statically assigned to cores and are scheduled according to fixed priorities. No task migration is allowed between cores at runtime.

To avoid contention on the shared memory, at most only one memory phase (read phase or write phase) can be allowed at the same time. The arbitration of memory access is based on the priority of tasks. The pending memory phase with the highest priority out of all cores will be granted access.

3.3.2 Preemption model

Preemptions are not allowed in the read phase and write phase of a task, and the execution phase may be preempted.

In this thesis, the **OMPM** method introduced in Section 2.2.2 is used for realizing preemptions in execution phases. When preemption occurs, the execution of the preempted task is paused with its intermediate data kept in the core-local memory, and the preempting task then enters the reading phase. The preempting task is also possible to be preempted by another task, provided that sufficient space remains in the core-local memory, i.e., $\sum_{\tau_k \in \mathcal{A}} M_k + M_p \leq S$, where $\mathcal{A}$ denotes the set of tasks currently kept in the core-local memory, and τ_p is the newly preempting task.

3.3.3 Limited preemption

To limit unnecessary preemptions, preemption thresholds introduced in Section 2.3.1 are used in this thesis.

Each task τ_i is assigned a preemption threshold θ_i , where

$$\theta_i \geq P_i.$$

Preemption of τ_i by τ_j during the execution phase is possible only if

$$P_j > \theta_i,$$

which means that the nominal priority of the preempting task exceeds the preemption threshold of the current running task. Otherwise, preemption will not occur.

Chapter 4

Methodology

The purpose of this chapter is to provide an overview of the research method used in this thesis. Engineering-related content is included in this chapter and will be specifically explained. This includes the stepwise procedures for solving the research problem, the designated platform for implementation, resulting data acquisition and analysis, etc.

4.1 Research process

Focusing on the research question mentioned in Chapter 1, the research process of this thesis is designed to achieve the two research goals, namely the implementation of an **RTOS** kernel that conforms to the theoretical model and the evaluation of the overheads of the preemption approach. Approximately, the implementation work takes 70% of the workload in this thesis, and the evaluation work takes the rest 30%. Several essential steps are included in the research process.

- Step 1** Implement a basic **RTOS** kernel in a single-core environment. The kernel operates with a non-preemptive fixed-priority scheduler and is able to execute tasks that are compatible with the three-phase execution model. Context switch of tasks and the management of core-local memory are implemented in this step.
- Step 2** Extend the **RTOS** kernel to a dual-core environment. Arbitration of shared memory between cores is implemented in this step. The **RTOS** kernel still remains fully non-preemptive and serves as the baseline for further evaluation.

- Step 3** Introduce the preemption approach to the **RTOS** kernel. The implementation of limited preemption strictly follows the theoretical model of **OMPM** and preemption thresholds. The kernel is able to operate in either fully non-preemptive mode or limited-preemptive mode, decided by a config file.
- Step 4** Evaluate the runtime overheads. Selected benchmark applications that are transformed for the three-phase task model, as well as synthetically generated applications, are used. The runtime overheads of introducing preemption are evaluated by comparing the performance of the **RTOS** kernel in both modes.

4.2 Engineering methodology

This section explains the engineering methodology of this thesis, including the details of implementing the expected **RTOS** kernel and evaluating the overheads of introducing the preemption approach.

4.2.1 Hardware platform to be used

In this thesis, the STM Stellar SR5E1E7 microcontroller [18] is used as the designated hardware platform. This is a **COTS** dual-core platform with two 32-bit Arm Cortex-M7 cores, and has 32KB instruction **Tightly Coupled Memory (TCM)** and 64KB data **TCM** for each core that can be used as the core-local memory. 256KB system SRAM can be accessed by both cores and is used as the shared memory. The official **Integrated Development Environment (IDE)**, Stellar Studio, is used for developing on the designated hardware. A Stellar Link debugger on the extension board is used for downloading binary executables and debugging.

4.2.2 RTOS kernel design

An **RTOS** that conforms to the theoretical model of the limited-preemptive three-phase execution model is expected to be implemented in this thesis.

A partitioned fixed-priority scheduler is implemented for the **RTOS** kernel. Tasks are assigned statically to processor cores offline and are not allowed to migrate between cores at runtime, and tasks are scheduled with fixed priorities. This is to minimize the complexity of the kernel, and focus on the part of implementing limited preemption.

Priority-based arbitration of memory phases between cores is implemented in the **RTOS** kernel. When contention on shared memory occurs, the arbitrator grants access according to the priority of tasks. The task with higher priority is permitted to access, while the task with lower priority is deferred until the shared memory is available.

To introduce preemptions in the **RTOS** kernel, the method of **OMPM** is used. This variant is chosen as it has been shown to work better with partitioned schedulers. The intermediate data of the preempted task is kept in the core-local memory, and can be resumed on the same core with minimal additional costs after the preemption with the method of **OMPM**.

For the implementation of limited preemption, the method of preemption thresholds is used in this thesis. This choice allows the implementation to closely follow the theoretical model of **OMPM**, and enables a controlled evaluation of overheads without modifying the structure of tasks.

4.2.3 Overhead evaluation

The overheads caused by introducing preemptions are evaluated by comparing the performance of the **RTOS** kernel in both fully non-preemptive and limited-preemptive modes.

Scheduling events are stored in a buffer in the core-local memory. By analyzing the difference between introducing preemptions and not, it is possible to evaluate the execution time overhead.

This thesis focuses on introducing preemption to the three-phase execution model, and does not include the part of converting an arbitrary task into the form of the three-phase execution model. Benchmark applications used for evaluations are specially designed. They strictly conform to the form of the three-phase execution model and can execute on the **RTOS** kernel implemented in the former part of the thesis.

4.3 Evaluation framework

The evaluation framework of this thesis mainly focuses on determining whether the implementation of the expected **RTOS** kernel works as expected. Scheduling events are recorded into a buffer of the core-local memory, and the evaluation work is based on the analysis of the scheduling behaviors. The key criteria of the evaluation framework include:

1. The correctness of basic functions in the **RTOS** kernel. Tasks should be

assigned to the predefined cores and should be scheduled in the correct order of fixed priorities. In the dual-core environment, arbitration of the shared memory should correctly follow the priority of tasks.

2. The implementation of the preemption approach. Preemptions should occur when they are supposed to be, and the preemption behavior conforms to the theoretical model of **OMPM**. Execution of the preempted task should be able to resume correctly after the execution of the preempting task finishes.
3. The limit to preemptions. The method of preemption thresholds should be implemented to realize the approach of limited preemption. When preemption thresholds are properly set, preemptions should be correctly limited. Only tasks with priority higher than the current preemption threshold can successfully perform preemptions.

4.4 Assessing reliability and validity of the data collected

The assurance of reliability and validity of methods and data involved in this thesis is an essential concern. This includes the correctness evaluation of the desired **RTOS** kernel implementation and the measurement of the overheads.

4.4.1 Validity

The scheduling events are directly recorded into the core-local memory, and can be analyzed afterwards. The behaviors of the **RTOS** kernel are directly reflected, and the correctness of the implementation can be verified by referring to the theoretical model. This provides an accurate evaluation of the implementation of the expected **RTOS** kernel.

The overheads of introducing limited preemption are evaluated through direct measurements of the **RTOS** kernel as well. Time consumption can be quantitatively analyzed from the records, which gives an accurate measurement of the required aspects.

4.4.2 Reliability

The tracing mechanism runs consistently when the **RTOS** kernel operates. The behaviors and consumptions of the **RTOS** kernel are recorded in the same

format for each run, and can be used reliably for analyzing the correctness of the **RTOS** kernel, as well as the runtime overheads.

The tracing data are collected under multiple runs in identical experimental settings. Abnormal values are discarded, and the variations of valid data across runs remain within an acceptable range, supporting the reliability of the data collected.

Chapter 5

Implementation

In this chapter, we present how the **RTOS** that conforms to the theoretical model is implemented in detail. The entire implementation can be divided into two parts, namely implementing the expected **RTOS** on a single-core side and migrating it to a dual-core environment.

5.1 Implementation on a single-core side

In this section, we introduce how the implementation is carried out on a single-core side. In this part, it is expected to have an **RTOS** that supports the execution of three-phase tasks, and is able to perform limited preemption during execution phases, following the method of **OMPM** and preemption thresholds. This includes supporting concurrent task executions, managing core-local and shared memory access, and implementing limited preemption with **OMPM** and preemption thresholds.

5.1.1 Concurrent task execution

Similar to any common **RTOS**, the **RTOS** kernel implemented in this thesis needs to support multitasking, i.e., the concurrent execution of tasks. To achieve this, two principal parts are the context switch among tasks and the blocking mechanism.

5.1.1.1 Context switch

Context switch is the process of suspending the execution of a running task and resuming another by saving and restoring its state [19]. By utilizing this, it becomes possible to allow multiple tasks to share a single CPU.

On the Cortex-M platform, the context switch can be done with the help of storing and resuming the **Stack Pointer (SP)** and the task stacks of different tasks. On the Cortex-M7 platform, which is used in this thesis, the stack pointer is subdivided into the **Main Stack Pointer (MSP)** and the **Process Stack Pointer (PSP)**. In this thesis, **MSP** is used in the handler mode for interrupts, and **PSP** is used in thread mode for running tasks. Thus, context switch in this thesis aims at handling **PSP**.

When a context switch happens, the current value of **PSP** is stored in the **Task Control Block (TCB)** of the current running task, and **PSP** will be set to the restored **PSP** value of the next task's **TCB**. Listing 5.1 shows the data structure of **TCB** used in this thesis. The value of **PSP** is stored in the first element of **TCB**. Other elements in **TCB** will be introduced later in this thesis.

Listing 5.1 TCB data structure

```
typedef struct _PdOSTaskControlBlock
{
    void      *psp;           // stack pointer
    uint32_t   wkUpTime;      // next wake-up time
    uint8_t    prio;          // priority
    uint8_t    threshold;     // preemption threshold
    uint8_t    phase;         // current phase
    uint8_t    preserve;      // align to 4 bytes
    uint32_t    memStartIndex; // start index in memory pool
    uint32_t    memSize;      // memory size
} PdOSTaskControlBlock;
```

We define a *task handle* as the pointer to the **TCB** of a task, i.e., `typedef PdOSTaskControlBlock *PdOSTaskHandle`. We use global variables `currTask` and `nextTask` to denote the current running task and the next task.

When entering an interrupt, the Cortex-M hardware automatically pushes register R0-R3, R12, LR, PC, and xPSR to the current stack to preserve the execution context. Register R4-R11 needs to be handled manually in the context switch, as these are callee-saved registers according to the AAPCS standard [20]. In addition, we disable the **Floating Point Unit (FPU)** to simplify the usage of the stack frame, that is, to avoid handling additional floating-point registers, **Floating-Point Status and Control Register (FPSCR)**.

Context switch needs to be carried out in the handler mode, and the PendSV interrupt on Cortex-M platforms is the most suitable place for it.

Since context switch involves handling registers that may also be used by the compiled assembly code of C functions, the PendSV handler needs to be directly written in assembly instead of in C. Listing 5.2 shows the key logic of context switch written in assembly.

Listing 5.2 Key logic of context switch

```

/* save context */
LDR    R2, =currTask
LDR    R1, [R2, #0] /* R1 = currTask */
MRS    R0, PSP      /* R0 = current PSP */

STMDB  R0!, {R4-R11} /* save R4-R11 according to PSP */
STR    R0, [R1, #0] /* update currTask->psp */

/* restore context */
LDR    R2, =nextTask
LDR    R1, [R2, #0] /* R1 = nextTask */

LDR    R0, [R1, #0] /* R0 = nextTask->psp */
LDMIA  R0!, {R4-R11} /* restore R4~R11 according to PSP */

MSR    PSP, R0      /* update PSP */

```

When expecting to switch to another task, set global variable `nextTask` to the handle of the next task and trigger the PendSV interrupt. Afterwards, the context switch will be carried out in the PendSV interrupt, and the CPU will be switched to execute the next task.

5.1.1.2 Blocking mechanism

In this thesis, we consider tasks that are periodic, which means that tasks execute repeatedly at regular intervals [21]. In each period, a task executes for a fixed time and is blocked during the rest of the time. Tasks block themselves by the end of execution code, and set the next time of waking up to execute periodically. To achieve this, the **RTOS** needs to implement a blocking mechanism.

Task sets

We maintain two task sets, namely the ready set and the blocked set. The ready set contains tasks that are ready to execute, and the blocked set contains tasks that are blocked and are still waiting for the next wake-up time. When a task finishes execution in a period, it blocks itself. The **RTOS** kernel updates its next wake-up time and moves the task from the ready set to the blocked set. When it is the wake-up time of the blocked task, the **RTOS** kernel moves the task from the blocked set to the ready set. Only tasks in the ready set have the possibility to be chosen to execute by the scheduler.

To simplify the management of task sets, we require that each task has a unique priority number, and use priority to denote the task. Under this restriction, it becomes easy to maintain the ready set and the blocked set with bitmaps. We require the priority of tasks between 1 and 32, which corresponds to bits 0-31 in a 32-bit unsigned integer variable. Thus, we have variables with type `uint32_t` named `readySet` and `blockedSet`.

Figure 5.1 shows an example of the task sets using bitmaps. In this example, variable `readySet` has the value `0b00100100` and variable `blockedSet` has the value `0b01000001`. It can be referred to that the tasks with priorities 6 and 3 are ready to execute, and tasks with priorities 7 and 1 are currently blocked, and the **RTOS** has 4 tasks in total.

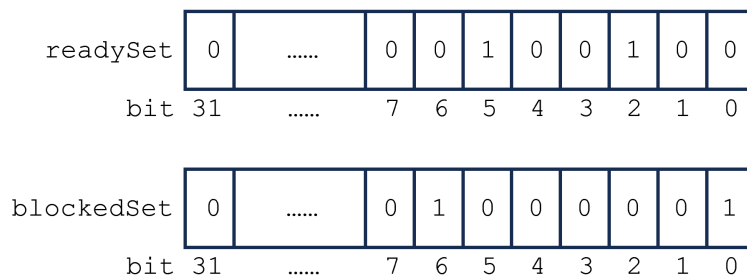


Figure 5.1: Example of task sets using bitmaps

Priority 0 is preserved for the idle task in this thesis. When there is no task ready, the idle task, which does nothing, will be chosen to execute.

System time management

To support the timing requirements of tasks, the **RTOS** kernel needs to maintain a system time variable, which is a 32-bit unsigned integer global variable `sysTime` in this thesis. We use the systick interrupt to maintain the

system time of the **RTOS** kernel. The frequency of the systick is set to 1000Hz, i.e., having a systick at exactly every millisecond.

When a task blocks itself, the kernel calculates its next wake-up time by adding the previous wake-up time and the period of the task, and writes the next wake-up time to element `wkUpTime` in the task's **TCB**. In the case that the task has already missed its deadline, the **RTOS** will not move the task to the blocked set, and instead keeps it in the ready set.

In the systick interrupt, the value of variable `systemtime` is added by 1. We then check whether there are any tasks that should be woken up by comparing the current value of system time and the next wake-up time of tasks recorded in their **TCBs**. Overflow of unsigned integers should be carefully considered, since the value of `systemtime` will wrap back to 0 after exceeding the maximum value of the 32-bit unsigned integer. Rather than simply comparing the value of `systemtime` and `wkUpTime`, we subtract them and convert the result to a signed integer and compare it with 0. That is, use `if ((int32_t)(systemtime - h->wkUpTime) >= 0)` instead of `if (systemtime >= h->wkUpTime)`, as the latter cannot properly handle overflow of unsigned integers.

5.1.2 Memory management

Management of shared memory and core-local memory is one of the key topics in this thesis. The principle of the three-phase execution model is to keep tasks operating on the core-local memory and avoid contention for shared memory during execution phases. To achieve this, we use a specially modified memory layout and carefully design the management of memory usage in this thesis.

5.1.2.1 Memory layout configuration

The **RTOS** kernel itself needs to operate on the core-local memory and avoid accessing the shared memory to conform to the theoretical model. In this way, the vast majority of contention for shared memory is caused by the tasks, and the **RTOS** kernel does not introduce extra influences.

To do so, modifications to the linker script are needed. We use a customized linker script in this thesis, modified from the default linker script provided in the **Software Development Kit (SDK)**. Listing 5.3 shows key parts of the linker script after modifications.

Listing 5.3 Key parts of the linker script

```
.text : ALIGN(16) SUBALIGN(16)
{
    *(EXCLUDE_FILE(*systick*.o *PredictOS*.o) .text)
    *(EXCLUDE_FILE(*systick*.o *PredictOS*.o) .text.*)
    *(EXCLUDE_FILE(*systick*.o *PredictOS*.o) .rodata)
    *(EXCLUDE_FILE(*systick*.o *PredictOS*.o) .rodata.*)
    /* the reset omitted */
} > EMBED_NVM

.itcm ORIGIN(EMBED_ITCM) : ALIGN(16)
{
    /* the rest omitted */
    *systick*.o(.text*)
    *PredictOS*.o(.text*)
    *systick*.o(.rodata*)
    *PredictOS*.o(.rodata*)
} > EMBED_ITCM AT> EMBED_NVM
```

In the customized linker script, we replace text (`.text*`) and read-only data (`.rodata*`) relevant to the **RTOS** kernel from the default `.text` section to the `.itcm` section, which corresponds to the instruction **TCM** on the platform. In this way, the instructions relevant to the **RTOS** kernel are placed in the instruction **TCM** and can remain independent for instruction fetch. In particular, `systick` is provided as a driver in the **SDK**, and as a result, we need to place the `systick` driver into instruction **TCM** as well, since the **RTOS** kernel relies on `systick` to operate.

In addition, the **RTOS** kernel heavily relies on the handler mode, in which **MSP** and the main stack are used. To ensure that the **RTOS** kernel does not access shared memory during operation, we need to place the main stack in the core-local memory as well, which is the data **TCM** in this thesis. Listing 5.4 shows how the main stack is placed into the data **TCM** in the linker script.

Listing 5.4 Place main stack into the data **TCM**

```
.mstack (NOLOAD) :
{
    /* contents omitted */
} > EMBED_DTCM
```

Another modification we need to consider is about preserving memory space in the main memory, which is used for memory management in the following part of the thesis. By subtracting $0x4000$ from the heap end, we preserve a slice of memory with length 16k bytes on each core. That is, we have `__heap_end__ = ORIGIN(EMBED_RAM) + LENGTH(EMBED_RAM) -  $0x4000$` in the linker script. Memory address between `__heap_end__` and the end of SRAM on each core can be used freely by the **RTOS** kernel.

Figure 5.2 shows the memory layout of SRAM on both cores using the customized linker script, with specific memory addresses marked out. Note that in this thesis, dynamic memory allocation is not used. As a result, the heap region in SRAM does not actually grow. Also, section `.pstack` is still placed in SRAM rather than in the data **TCM**. This does not affect the memory independency of task execution, as the **PSP** will be redirected to the task stacks which are explicitly placed in the data **TCM** by using `__attribute__((section(".dtcm")))`, and section `.pstack` will not actually be used by tasks.

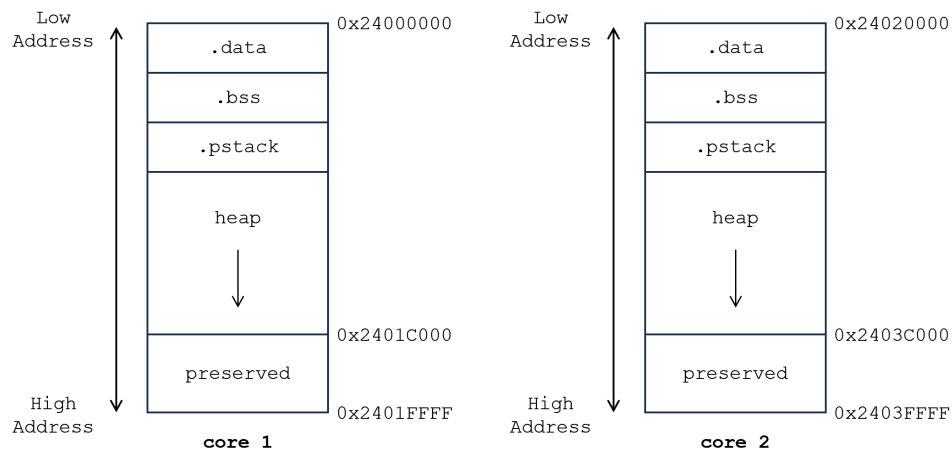


Figure 5.2: Memory layout of SRAM

5.1.2.2 Management of core-local and shared memory

According to the theoretical model, instructions and data of tasks should be loaded into the core-local memory to avoid memory contention for shared memory. However, on Cortex-M platforms, executables are typically linked to fixed addresses at build time. Dynamically moving task instructions between

shared memory and core-local memory requires additional runtime relocation support and **Position-Independent Code (PIC)** generation, which are typically unavailable on bare-metal Cortex-M platforms and significantly exceed the scope of this thesis. In addition, the dynamic moving of task stacks at runtime also requires work that exceeds the scope of this thesis. As a result, the implementation of this thesis will focus on the handling of runtime data of tasks, with instructions and stack placed statically in the core-local memory.

Task structure

To simplify the implementation and focus on the theoretical model of three-phase execution, we consider tasks accessing independent runtime data. That is, each task has a slice of memory space that only belongs to itself and cannot be accessed by other tasks. In this way, race conditions, which are not the main focus of this thesis, can be eliminated.

Listing 5.5 gives the pseudocode of a task in this thesis. During each iteration of the task loop, the task enters the read phase by explicitly calling the read **Application Programming Interface (API)** provided by the **RTOS** kernel, which copies its data from the shared memory to the core-local memory and returns the starting address of the task's data slice. After that, the task will operate on its data slice, which corresponds to the execution phase. Finally, the task enters the write phase by explicitly calling the write **API** provided by the **RTOS** kernel that copies the data slice from core-local memory back to the shared memory.

Listing 5.5 Pseudocode of a task

```
loop forever

    startAddress := PdOS_read()
    execute(startAddress)
    PdOS_write()

    delay until next iteration

end loop
```

Shared memory management

To implement the aforementioned **APIs** for moving task's runtime data between core-local memory and shared memory, a special memory management mechanism needs to be designed. We use a designated memory area in the preserved section in SRAM as a memory pool in the shared memory. Each task is assigned to a particular slice in the memory pool, in which its runtime data will be stored. Since tasks keep running and will never be deleted after being created in this thesis, the allocation of memory slices in the memory pool is monotonous, i.e., a new task is always assigned with the memory right after the previously created task, and the memory slice will never be freed.

To allocate memory slices from the memory pool, the **RTOS** kernel needs to maintain a variable `mainMemFreeIndex`, which always points to the first available position in the memory pool. When a task is created, `mainMemFreeIndex` is added by the size of the runtime data that the task requires, and this slice in the memory pool is assigned to the task. The starting index in the memory pool of the slice is recorded in the `memStartIndex` element in the task's **TCB**, along with its size recorded in element `memSize`, for later usage.

Figure 5.3 shows an example of allocation in the memory pool. In this example, tasks 1-4 have already been created, with each of them assigned to a specific slice in the memory pool. `mainMemFreeIndex` currently points right after the slice of task 4, which is the first available position in the memory pool. When task 5 is about to be created, we add `mainMemFreeIndex` by the size of the runtime data that task 5 requires, and move it to the new position after allocating this memory slice.

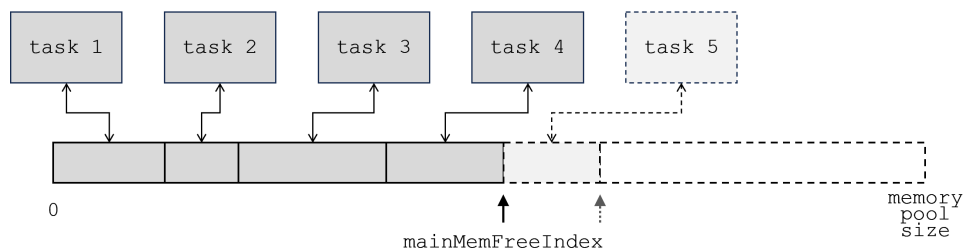


Figure 5.3: Example of allocation in the memory pool

Core-local memory management

The management of core-local memory follows the theoretical model of **OMPM**, where intermediate data will be kept in the core-local memory when a task is preempted. In this thesis, we use a memory buffer in the data **TCM**, which serves as the core-local memory. Unlike the memory pool in the shared memory that allocates monotonously, memory in the core-local memory buffer is managed in a **Last-in, First-Out (LIFO)** manner.

The **RTOS** kernel maintains a variable `localMemFreeIndex`, which always points to the first available position in the core-local memory buffer. When a task is about to run and calls the read **API**, regardless of the existence of preempted tasks, its slice in the memory pool of shared memory will be copied to the core-local memory buffer and placed at the position where `localMemFreeIndex` points to if there is enough space. Afterwards, `localMemFreeIndex` will be added by the size of the slice, similar to the behavior in the memory allocation in the shared memory. The read **API** of the **RTOS** kernel returns the starting address of the slice in the core-local memory, so that the task can operate on it. When the task finishes executing and calls the write **API**, its slice in core-local memory will be copied back to its slice in the memory pool of shared memory, and `localMemFreeIndex` will be subtracted by the size of the slice. Note that the read **API** needs to return the starting address of the task's slice in the buffer in core-local memory every time, as the position of the slice may vary among executions.

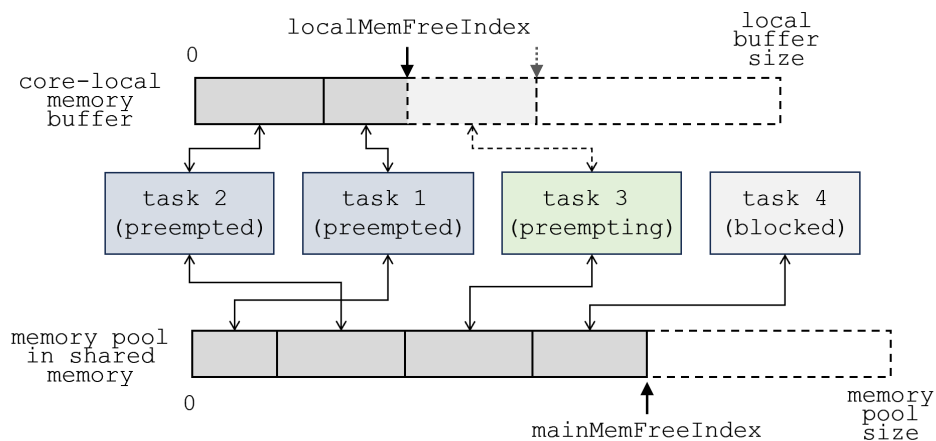


Figure 5.4: Example of managing core-local memory

Figure 5.4 shows an example of managing core-local memory. In this example, tasks 1-4 have already been created and assigned with slices in the

memory pool in shared memory. Task 2 has been loaded into the core-local memory first, and was preempted by task 1 afterwards. Currently, task 1 is about to be preempted by task 3, which has not been loaded into the core-local memory. During the read phase of task 3, its slice in the memory pool in shared memory will be copied into the buffer in core-local memory, and `localMemFreeIndex` will be added by the size of the slice. The read **API** returns the starting address of the slice in the buffer, so that task 3 can operate on its slice with slices of task 1 and task 2 kept in the core-local memory, which conforms to the method of **OMPM**. When task 3 finishes executing, it calls the write **API** to copy its slice from the buffer in core-local memory back to its slice in the memory pool in the shared memory. `localMemFreeIndex` then resumes to the position before task 3 preempts task 1.

5.1.3 Scheduler design

The design of the scheduler is another critical topic of this thesis. In section 5.1.1.1, we have implemented the mechanism of context switching among tasks. By setting the global variable `nextTask` to the handle of the next task and triggering the `PendSV` interrupt, the CPU will be switched away from the current task and execute the next task. The responsibility of the scheduler is to find the proper `nextTask` that conforms to the theoretical model, including the restrictions of task phases, priorities, thresholds, and memory usage, etc.

The scheduler is implemented as an internal function in the **RTOS** kernel, and it will check whether there is any available task that should switch to and set the global variable `nextTask` when being called. The scheduler can be called either by the **RTOS** kernel in a scheduler tick or by the task itself when it tries to block itself via **APIs** provided by the **RTOS** kernel. For the former case, a *preemption* happens. And for the latter case, a *CPU yield* is carried out. The scheduler needs to handle both cases properly.

5.1.3.1 Stack invariant under OMPM

Before introducing the implementation of the scheduler, we need to introduce the stack invariant property when using **OMPM**, which is critical to the implementation.

Consider task execution on a single core under fixed-priority scheduling and preemption thresholds, and with core-local memory arranged following the method of **OMPM**. Each task τ_i has a nominal priority P_i and a preemption threshold $\theta_i \geq P_i$. At any instant t , we define the *preemption stack* on the core

as the ordered sequence

$$\mathcal{S}(t) = [\tau^{(1)}, \tau^{(2)}, \dots, \tau^{(k)}],$$

which contains all tasks currently in the execution phase, ordered by the time that they entered the execution phase, and the top element $\tau^{(k)}$ is the task currently executing. The stack can be modified via push or pop operations. A push is carried out when a task preempts the current running task and enters its execution phase; a pop is carried out when the preempting task finishes its execution.

Here, we introduce the definition of the stack invariant:

Definition: At any instant t , for every adjacent pair $\tau^{(i)}, \tau^{(i+1)} \in \mathcal{S}(t)$,

$$P_{\tau^{(i+1)}} > \theta_{\tau^{(i)}} \geq P_{\tau^{(i)}}.$$

Proof:

Base case: The initial stack is empty, so the stack invariant holds vacuously.

Case 1 (pop): Removing $\tau^{(k)}$ leaves a prefix of $\mathcal{S}$. The remaining part satisfies the stack invariant by hypothesis.

Case 2 (push): A new task τ_{new} preempts the running task $\tau^{(k)}$. The restriction of preemption threshold requires $P_{\tau_{\text{new}}} > \theta_{\tau^{(k)}}$. Combined with $\theta_{\tau^{(k)}} \geq P_{\tau^{(k)}}$, the new adjacent pair satisfies the stack invariant.

Hence, the stack invariant always holds during the push and pop operations.

5.1.3.2 Preemption

Preemptions in an **RTOS** are implemented by the kernel checking whether the CPU should be switched away from the current task and be set to run a preempting task at every tick interrupt [22]. In this thesis, the scheduler will be called intentionally in every systick interrupt when preemptions are allowed. When the scheduler decides not to set `nextTask` in this case, preemption will not occur. Due to the restrictions in this thesis, such as phased tasks, local memory usage, preemption thresholds, etc., special considerations need to be included.

Listing 5.6 gives the pseudocode of the preemption path of the scheduler. The scheduler first checks that the current task is not in its memory phases, and then calls a helper function that finds the task with the highest nominal priority and can be fitted in the core-local memory as well. Listing 5.7 gives

the pseudocode of the helper function for finding the candidate task. The scheduler then checks whether the candidate task differs from the current running task, and has a nominal priority greater than the preemption threshold of the current running task that the candidate task is about to preempt. If all conditions are satisfied, the scheduler will set `nextTask` to the candidate task and trigger the `PendSV` interrupt to perform a context switch.

Listing 5.6 Pseudocode of the preemption path

```

procedure scheduler_in_preemption_path
    if currTask is in read/write phase then
        return
    end if
    candidateTask := find_highest_priority_available_task()
    if candidateTask == currTask then
        return
    endif
    if candidateTask.priority <= currTask.threshold then
        return
    end if
    nextTask := candidateTask
    trigger PendSV interrupt
end procedure

```

It should be specially noticed that the candidate task chosen according to the highest nominal priority will never be in the execution phase. This is guaranteed by the stack invariant proven in 5.1.3.1. Since the current running task is still in the ready set, the selection of the candidate task means it has a higher nominal priority than the current running task, i.e., $P_{\text{candidate}} > P_{\text{current}}$. If the candidate task is in the execution phase, it means that it is already in the preemption stack. However, with the current running task in its execution phase located at the top of the stack, the stack invariant implies that the current running task has the highest nominal priority among all tasks in the stack, which means $P_{\text{candidate}} < P_{\text{current}}$, and contradicts with the previous inequality. Thus, the candidate task cannot be in the execution phase. The comparison between the candidate task's nominal priority and the current running task's preemption threshold in listing 5.6 is sufficient.

Listing 5.7 Pseudocode of the function finding the highest-prioritized available task

```

function find_highest_priority_available_task
    tmpReadySet := readySet
    while tmpReadySet is not empty do
        maxPrioTask := highest-priority task in tmpReadySet
        if maxPrioTask is already in execution phase then
            return maxPrioTask
        end if
        if local memory has enough space for maxPrioTask then
            return maxPrioTask
        end if
        remove maxPrioTask from tmpReadySet
    end while
    return idle task
end function

```

5.1.3.3 CPU yield

When a CPU yield happens, the current running task blocks itself and intentionally calls the scheduler to let another task run. In this case, the scheduler needs to find the proper task as the next task. The task preempted by the currently running task, or a newly awakened task that has not been set to run, will be chosen. That is,

$$P_{\text{next}} = \max \left(\max_{\tau_i \in \text{readySet}} P_i, \max_{\tau_j \in \text{preemptionStack}} \theta_j \right)$$

Listing 5.8 gives the pseudocode of the CPU yield path of the scheduler, and listing 5.9 gives the pseudocode of the helper function for finding the top task in the preemption stack.

In the CPU yield path, the scheduler first checks that the current task is not in its memory phases, similar to the case of the preemption path. Then, the scheduler finds the highest-prioritized available task in the ready set by using the helper function in listing 5.7, and finds the top task in the preemption stack by using the helper function in listing 5.9. It compares the nominal priority of the former and the preemption threshold of the latter, and selects the one with the highest value as the next task.

Listing 5.8 Pseudocode of the yield path

```

procedure scheduler_in_yield_path
    if currTask is in read/write phase then
        return
    end if
    candidateInReadySet := find_highest_priority_available_task()
    candidateInStack := find_top_task_in_preemption_stack()
    nextTask := max{candidateInReadySet.priority,
                    candidateInStack.threshold}
    trigger PendSV interrupt
end procedure

```

Listing 5.9 Pseudocode of the function finding the top task in the preemption stack

```

function find_top_task_in_preemption_stack
    tmpReadySet := readySet
    maxThresTask := idle task
    while tmpReadySet is not empty do
        tempTask := highest-priority task in tmpReadySet
        if tempTask is in execution phase then
            if tempTask.threshold > maxThresTask.threshold then
                maxThresTask := tempTask
            end if
        end if
        remove tempTask from tmpReadySet
    end while
    return maxThresTask
end function

```

5.2 Migration to the dual-core environment

In this section, we introduce how the migration to a dual-core environment is carried out. The **RTOS** is expected to operate in parallel on both cores, with each having its fixed task set. When contention to shared memory occurs, access should be arbitrated according to the priority of the current executing tasks to have predictable behaviors. This part includes the synchronization of the two cores and the implementation of the arbiter.

5.2.1 Synchronization

On the dual-core platform used in this thesis, the systick of the two cores operates independently. In the arbiter design introduced later in section 5.2.2, each core has a share of the arbiter running on its RTOS kernel, deciding which core is allowed to access the shared memory before entering memory phases. This arbitration mechanism relies on the synchronization of the systick on the two cores, as once a core has been granted access to the shared memory, the other core, even with a higher priority task running, cannot preempt it.

Figure 5.5 shows an example of how the arbitration mechanism will fail when the systick on the two cores is not well synchronized. In this example, core 1 has a low-priority task τ_1 , and core 2 has a high-priority task τ_2 running respectively. Both tasks are released at time 0, based on the system clock on their cores, attempting to enter their read phase. According to the theoretical model, core 2 should win the arbitration, since τ_2 running on it has a higher priority. However, the systick on the two cores is not well synchronized. When τ_2 on core 2 tries to enter its read phase at time 0, according to core 2's system clock, τ_1 on core 1 has already occupied the access to the shared memory since no task was waiting at time 0 according to core 1's system clock. In this case, τ_2 on core 2 with a higher priority is blocked by τ_1 on core 1 with a lower priority, which reveals the failure of the arbitration mechanism.

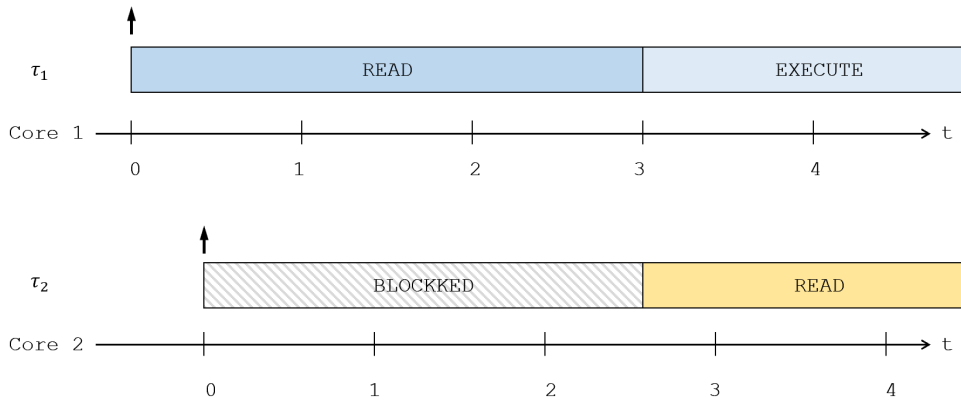


Figure 5.5: Example of arbitration failure when not well synchronized

To address this issue, the synchronization of the systick on the two cores is needed. On the dual-core platform used in this thesis, the two cores can share the same clock source [18]. This means that all the following ticks are well synchronized, as long as the first tick is synchronized. To synchronize the first systick on the two cores, the hardware semaphore can be used. The idea is to

block the start of the systick on both cores by using the hardware semaphore, and release at the same time.

Listing 5.10 Pseudocode of synchronization

```

procedure synchronization_on_core_1
    acquire hardware semaphore
    start core 2
    delay for a while
    release hardware semaphore
    start systick
end procedure

procedure synchronization_on_core_2
    acquire hardware semaphore
    release hardware semaphore
    start systick
end procedure

```

Listing 5.10 shows the pseudocode of synchronization of systick on the two cores. Note that on the dual-core platform used in this thesis, core 2 is started by core 1 intentionally. During the synchronization process, core 1 first acquires the hardware semaphore before starting core 2. When core 2 starts, it tries to acquire the hardware semaphore too, and will be blocked. Core 1 waits for a while to make sure that core 2 has started and has been blocked. Then, core 1 releases the hardware semaphore and starts its systick. At the same time, core 2 successfully acquires the hardware semaphore and unblocks. It releases the hardware semaphore immediately and starts its systick. With this synchronization mechanism, the difference in the first systick between two cores can be minimized, and this time difference can be covered by using a short waiting window in the arbiter.

5.2.2 Arbiter design

According to the theoretical model of phased executions on multiple cores, the core with a high-priority task running always wins the arbitration for access to shared memory. In this thesis, we use a shared data structure placed in the shared memory to coordinate between the two cores.

Listing 5.11 shows the data structure of the arbiter. In the data structure of the arbiter, element `req` refers to whether a core is requesting access to shared memory, and element `prio` corresponds to the priority of the current running

task on the cores, on which the arbitration is based. The arbiter is placed at a fixed position in the shared memory. In this thesis, we place it at a fixed address in the preserved area of core 2, mentioned in section 5.1.2.1.

Listing 5.11 Arbiter data structure

```
typedef struct _PdOSArbiterType
{
    volatile uint8_t req[2];    // request of two cores
    volatile uint8_t prio[2];  // priorities
} PdOSArbiterType;
```

Listing 5.12 shows the pseudocode of the arbitration logic. When a core tries to acquire access to the shared memory, it first writes the priority of the current running task in the corresponding `prio` element of the arbiter, and sets the corresponding `req` element afterwards. Then, it waits for a short while for the other core to write its request. If the other core is also requesting access to shared memory and has a higher priority, the current core will back off and wait until the other core finishes. Otherwise, it will be granted access to shared memory. In this thesis, we use the hardware semaphore here to represent the status of shared memory. Only one core can hold the hardware semaphore at a time and can have access to the shared memory. For the release of the arbiter, the core simply releases the hardware semaphore and resets its corresponding `req` element. After that, the waiting core can be unblocked and granted access to the shared memory.

Note that in the arbitration logic, element `prio` needs to be written before element `req`. This is to make sure that when the other core notices the request for it, the priority has already been set to the correct value, so that the comparison of priorities is valid. Also, we use a short waiting window for all cores to finish their registration. This is because the synchronization mechanism introduced in section 5.2.1 does not guarantee absolute synchronization, but instead minimizes the time difference between two cores to an acceptable level, which can be eliminated by using the waiting window. If a core finds the other core not registered by the end of the waiting window, it is considered that the other core is not requesting the shared memory, and the current core will be directly granted access.

One more thing to be noticed is that a back-off period is used when a core loses arbitration and checks the status of another core. The purpose of using it is to lower the frequency of bus access, since the arbitration logic itself needs to access the shared memory, which is also used by the memory phases of tasks

Listing 5.12 Pseudocode of arbiter

```

procedure arbiter_acquire
    ARBITER.prio[self] := currPrio
    ARBITER.req[self] := 1
    while in waiting window do
        if ARBITER.req[other] == 1 then
            break
        end if
    end while
    while ARBITER.req[other] == 1 and
        ARBITER.prio[other] > currPrio do
        back off for a while
    end while
    acquire hardware semaphore
end procedure

procedure arbiter_release
    release hardware semaphore
    ARBITER.req[self] := 0
end procedure

```

and may cause contention. In other words, contention for the shared memory between cores cannot be 100% eliminated, but the contention caused by the **RTOS** kernel itself can be minimized with the use of the back-off period, with contention caused by tasks totally eliminated.

Chapter 6

Evaluation

In this chapter, we present how the evaluation of the expected **RTOS** is carried out. This includes the tracing mechanism of scheduler events and the evaluation of the overheads.

6.1 Scheduling behaviors

Tracing and analyzing scheduling behaviors is one of the key evaluations of the **RTOS** kernel. In this section, we introduce how the scheduling events are recorded and analyze several representative scheduling behaviors of the **RTOS** kernel.

6.1.1 Logging mechanism

To trace the scheduling behavior of the **RTOS**, we need to implement a logging mechanism that records all scheduling behaviors into a circular buffer for later analysis. Since this thesis focuses on the three-phase execution model, the scheduling events we need to record are the entry and exit of phases. Also, the current system time and the release time of the current tasks need to be recorded, along with the scheduling events. Since we use unique priorities to denote tasks, we need to record the priority of tasks in the log as well.

Figure 6.1 shows the log format used in this thesis. Each log entry consists of two 32-bit words, with its first word recording the current system time. The following word records the duration of the current task at bits 31-16, the priority of the task at bits 15-8, and the scheduling event at bits 7-0.

Here, we do not directly record the release time of the task, since a timestamp needs to occupy 32 bits, which is too much for a log entry. Instead,

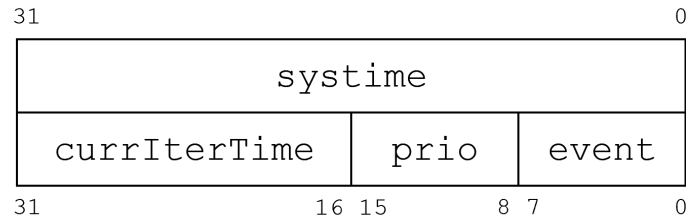


Figure 6.1: Log format

we record the difference between the current system time and the release time of the task, i.e., the duration of the current task. We restrict the period of a task to not exceed 65535 milliseconds, so that this difference can be represented by a 16-bit unsigned integer. Period exceeding 65535 milliseconds is still supported by the **RTOS** kernel using the blocking mechanism introduced in section 5.1.1.2, but the log recording will fail.

Due to the restriction of the three-phase execution model, the shared memory should only be accessed during the memory phases. The log recording mechanism should also follow this restriction, so we implement two log buffers, one in the core-local memory and the other in the shared memory. Scheduling events are instantly recorded in the log buffer in the core-local memory, and the contents of the local log buffer are copied to the log buffer in the shared memory during the write phases, together with the intermediate data of tasks.

When enough log has been recorded, we halt the **RTOS** kernel and copy the recorded log from the shared memory out to an external file by reading the designated memory area with the debugger. Afterwards, a Python script is used for parsing the log offline and displaying the scheduling behaviors in a visualized form. The following part of this section analyzes the scheduling behavior of the **RTOS** kernel under different conditions.

6.1.2 Scheduling analysis

With the help of the logging mechanism and the offline visualization tool, it is able to analyze the scheduling behaviors of the **RTOS** kernel under different conditions. Several representatives are introduced in the following.

6.1.2.1 Comparison with the theoretical scheduling example

In [7], an example task set is used to illustrate the scheduling behavior of a single-core **RTOS** kernel, which supports executing preemptive three-phase

tasks and uses preemption thresholds. The example task set consists of three tasks, namely $\tau_1 = (16, 10KB, 3, 5, 1, 2, 1)$, $\tau_2 = (24, 9KB, 2, 2, 1, 1, 1)$ and $\tau_3 = (32, 14KB, 1, 0, 3, 13, 1)$, where $\tau_i = (T_i, M_i, P_i, O_i, C_i^r, C_i^e, C_i^w)$ that corresponds to the theoretical model introduced in chapter 3. Preemption thresholds, 3, 3, and 1 are assigned to τ_1 , τ_2 , and τ_3 .

Figure 6.2 shows the theoretically analyzed scheduling behavior of the aforementioned example task set. It is important to note that τ_3 can be preempted during its execution phases by τ_1 and τ_2 , but τ_2 cannot be preempted by τ_1 , since τ_1 does not have a nominal priority greater than τ_2 's preemption threshold. For instance, τ_3 cannot be preempted by τ_2 at time 2 since the former is in its read phase, and τ_2 cannot be preempted by τ_1 at time 5 due to the restriction of preemption thresholds.

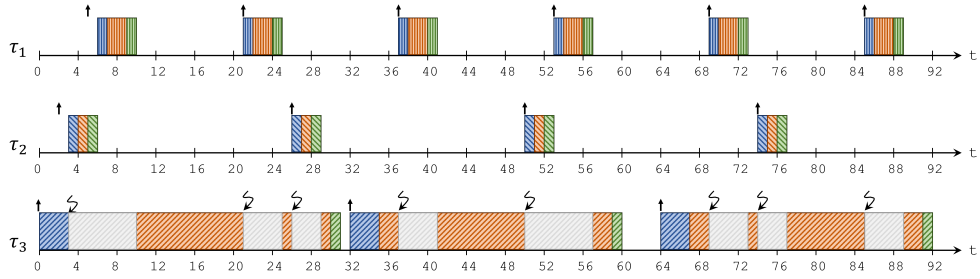


Figure 6.2: Theoretical scheduling behavior of the example task set

On the **RTOS** kernel implemented in this thesis, we try to reproduce the scheduling behavior of the example task set above. Since the tasks use dummy loop cycles for controlling time consumption and may not be absolutely accurate, we multiply the execution time of each phase by a factor of 10. Figure 6.3 shows the scheduling behavior of the **RTOS** kernel on the task set. Compared to the theoretically analyzed scheduling behavior in figure 6.2, it can be seen that the **RTOS** kernel works as expected on the example task set.

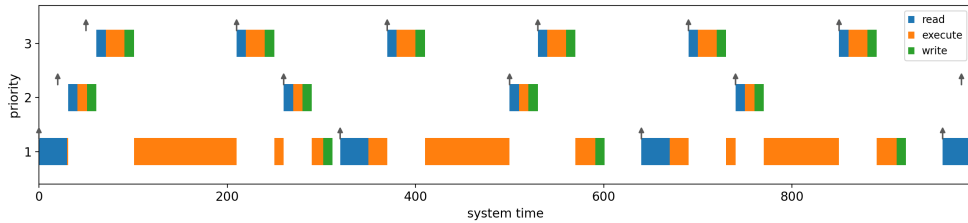


Figure 6.3: Actual scheduling behavior of the example task set

6.1.2.2 Fully-preemptive mode in execution phases

We consider a task set executing on a single core and consists of 3 tasks: $\tau_1 = (30, 100, 5, 0, 2, 5, 2)$, $\tau_2 = (130, 200, 3, 0, 6, 10, 5)$ and $\tau_3 = (200, 100, 2, 0, 8, 30, 10)$, where $\tau_i = (T_i, M_i, P_i, O_i, C_i^r, C_i^e, C_i^w)$, and the preemption threshold of each task equals their nominal priority. The size of the core-local memory buffer is set to 1000, so that the intermediate data of all 3 tasks can be placed in the core-local memory at the same time, i.e., memory restriction does not apply. In this case, τ_3 can be preempted during its execution phases by τ_2 , and τ_2 can be preempted during its execution phases by τ_1 .

Figure 6.4 shows the scheduling behavior of the **RTOS** kernel in fully-preemptive mode, with time ranging approximately from 1100 to 2000.

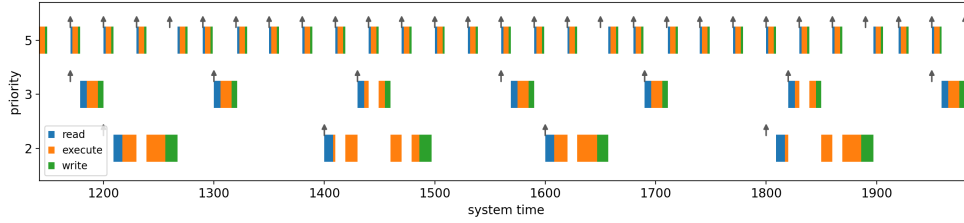


Figure 6.4: Scheduling behavior in fully-preemptive mode

We can see that in this log record, tasks with lower priority can always be preempted in their execution phases by tasks with higher priority, and the preemption can be nested. τ_3 is preempted by τ_2 at time 1430 and 1820, by τ_1 at time 1230, 1410, 1470, 1620, and 1860. τ_2 is preempted by τ_1 at time 1440 and 1830. Nested preemptions occur at time 1440 and 1830, where τ_3 is preempted by τ_2 and τ_2 is preempted by τ_1 , and the intermediate data of all 3 tasks are placed in the core-local memory at the same time. The sequence of preemption and resume is in a **LIFO** manner, i.e., τ_1 yields to τ_2 , and τ_2 yields to τ_3 after the nested preemption.

The scheduling is always based on the priority of tasks. When different tasks are released at the same time, the one with higher priority is always scheduled to run while others are blocked. At time 1170, 1560, and 1950, τ_1 and τ_2 are released at the same time, and τ_1 is scheduled to run. Similarly, at time 1200 and 1800, when τ_1 and τ_3 are released at the same time, τ_1 is scheduled to run.

Also note that memory phases of tasks cannot be preempted. At time 1260 and 1890, although τ_1 has been released, τ_3 is in its memory phase and cannot be preempted. τ_1 needs to wait until τ_3 exits its memory phase to preempt.

6.1.2.3 Limited-preemptive mode with preemption thresholds

In this section, we apply preemption thresholds, 5, 3, and 4, to τ_1 , τ_2 , and τ_3 in section 6.1.2.2. This means that the priority of τ_3 will be escalated to 4 during its execution phases and cannot be preempted by τ_2 with nominal priority 3, but can be preempted by τ_1 with nominal priority 5. Figure 6.5 shows the scheduling behavior of the RTOS kernel in limited-preemptive mode with preemption thresholds.

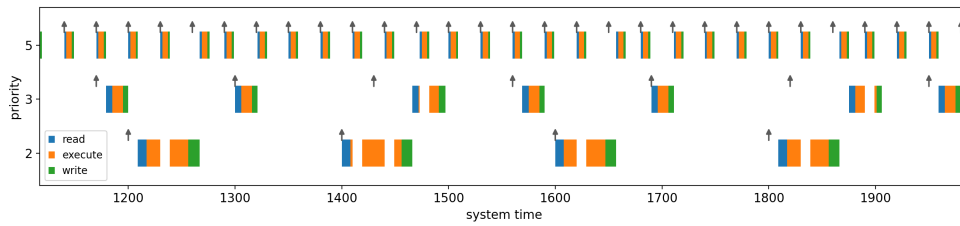


Figure 6.5: Scheduling behavior in limited-preemptive mode

We can see that τ_3 is preempted by τ_1 at time 1230, 1410, 1440, 1620, and 1830, τ_2 is preempted by τ_1 at time 1472 and 1890. However, τ_3 is never preempted by τ_2 due to the restriction of preemption thresholds. At time 1430 and 1820, τ_2 is released when τ_3 is in its execution phase, but preemption does not occur. Compared to the task set in section 6.1.2.2, τ_3 was preempted by τ_2 at these times when preemption thresholds were not applied.

Another important issue to be noticed is that, when τ_1 finishes its preemption to τ_3 at time 1449 and 1839, CPU yields to τ_3 instead of τ_2 , although the latter with a higher nominal priority has been released and is ready to execute at the moment. This conforms to the theoretical model, since τ_3 is in its execution phase and its priority is escalated to its preemption threshold of 4, while τ_2 has not been started and its nominal priority of 3 is used for comparison. This is the result of the well-designed logic introduced in section 5.1.3.3, which uses preemption thresholds for preempted tasks to compare. Otherwise, the CPU will yield to τ_2 instead of τ_3 if the comparison is simply based on nominal priorities, and will violate the theoretical model.

Also note that other features, such as priority-based scheduling and non-preemptive memory phases, still hold for the task set in this section, similarly to the one in section 6.1.2.2.

6.1.2.4 Preemption with insufficient core-local memory

In this section, we use the same task set as in section 6.1.2.2, and do not apply preemption thresholds. However, we set the size of the core-local

memory buffer to 300, so that the 3 tasks cannot be fitted into the core-local memory at the same time, and preemptions may be rejected due to this memory restriction. Figure 6.6 shows the scheduling behavior of the **RTOS** kernel when having insufficient core-local memory. Here we record the log with the time range approximately from 2100 to 3000, as this time range shows a more comprehensive scheduling manner than the time range 1100-2000 for the case of insufficient core-local memory.

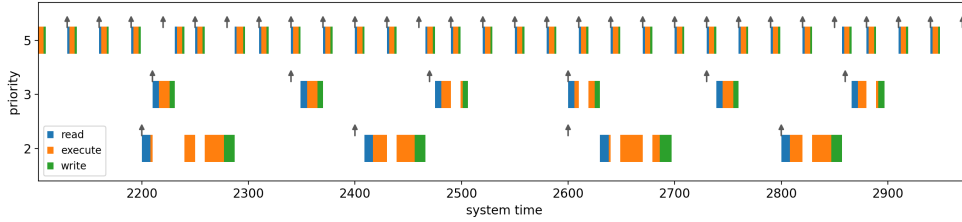


Figure 6.6: Scheduling behavior when having insufficient memory

In this case, we can see that low-prioritized tasks can still be preempted by high-prioritized tasks, but at most 2 tasks can execute concurrently due to the restriction of core-local memory space. τ_3 is preempted by τ_2 at time 2210, by τ_1 at time 2250, 2430, 2640, 2670, and 2820. τ_2 is preempted by τ_1 at time 2490, 2610, and 2880. However, nested preemptions, where τ_3 is preempted by τ_2 and τ_2 is preempted by τ_1 , do not occur. At time 2220, τ_2 has preempted τ_3 and is in its execution phase. Although τ_1 is released at this time, it cannot preempt τ_2 since there is not enough space for its intermediate data in core-local memory, and has to wait until time 2241 when τ_2 finishes executing. Also notice that at time 2241, τ_1 relays the preemption, and τ_3 is still being preempted. This conforms to the theoretical model where scheduling is based on priorities.

6.1.2.5 Fully non-preemptive mode

In this section, we use the same task set as in section 6.1.2.2, and set the **RTOS** kernel operating in fully non-preemptive mode. Preemptions will not occur in this case, and task switching only occurs when a task intentionally blocks itself.

Figure 6.7 shows the scheduling behavior of the **RTOS** kernel in fully non-preemptive mode. It can be seen that preemptions do not occur at all times, and τ_1 frequently misses its deadline.

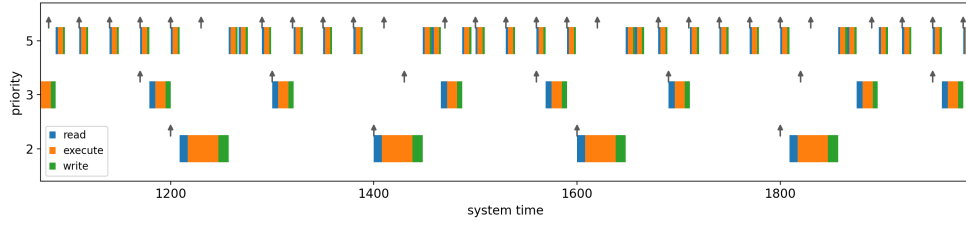


Figure 6.7: Scheduling behavior in fully non-preemptive mode

6.1.2.6 Dual-core scheduling behavior

In this section, we analyze the scheduling behavior in the dual-core environment. To make it easier for analysis, we simplify the tasks in section 6.1.2.2, decreasing task numbers and transferring them to a harmonic task set where a small hyperperiod exists. We apply the simplified tasks to the two cores, but with varying task priorities. Specifically, we consider a task set executing on dual cores and consisting of 4 tasks in total: $\tau_1 = (50, 200, 6, 0, 6, 10, 5)$, $\tau_2 = (200, 100, 2, 0, 8, 30, 10)$, $\tau_3 = (50, 200, 5, 0, 6, 10, 5)$, and $\tau_4 = (200, 100, 3, 0, 8, 30, 10)$, where $\tau_i = (T_i, M_i, P_i, O_i, C_i^r, C_i^e, C_i^w)$. Preemptions are allowed in execution phases, and preemption thresholds and memory restrictions do not apply. τ_1 and τ_2 are assigned to core 1, while τ_3 and τ_4 are assigned to core 2. Figure 6.8 shows the scheduling behavior of the **RTOS** kernel in the dual-core environment, with tasks on different cores represented by different colors.

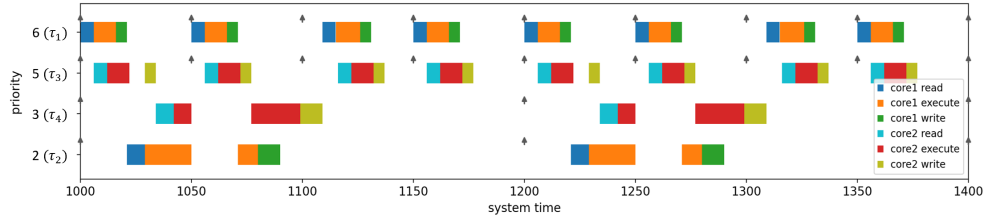


Figure 6.8: Scheduling behavior in the dual-core environment

The task set has a hyperperiod of 200, and the scheduling behavior of the **RTOS** kernel remains identical among different hyperperiods. Here, we take the hyperperiod from time 1000 to 1200 as an example for analysis.

- At time 1000, all the 4 tasks are released. The scheduler on core 1 selects τ_1 to run, and the scheduler on core 2 selects τ_3 , according to the priorities of each core's task set. These two tasks on two cores attempt to enter their read phase simultaneously, and the access to the shared memory is granted

to core 1 by the arbitration mechanism since τ_1 on it has a higher priority, and τ_3 on core 2 is blocked.

- At time 1006, τ_1 exits its read phase, and τ_3 is granted access to the shared memory and enters its read phase, which ends by time 1012.
- At time 1016, τ_1 on core 1 attempts to enter its write phase, and is granted access to the shared memory since τ_3 on core 2 is not requesting.
- At time 1021, τ_1 exits its write phase, and the CPU on core 1 yields to τ_2 . τ_2 attempts to enter its read phase at time 1021, and is granted access since τ_3 on core 2 is still in its execution phase and does not request access.
- At time 1022, τ_3 exits its execution phase and attempts to enter its write phase. Since the shared memory has already been granted to τ_2 at time 1021, τ_3 is blocked.
- At time 1029, τ_2 exits its read phase, τ_3 is granted access and enters its read phase.
- At time 1034, τ_2 exits its write phase, and the CPU on core 2 yields to τ_4 . τ_4 enters its read phase at time 1034, since the other core is not requesting.
- At time 1042, τ_4 exits its read phase. From time 1042 to 1050, τ_3 on core 1 and τ_4 on core 2 execute in their execution phases in parallel with no access to shared memory.
- At time 1050, a new iteration of τ_1 and τ_3 is released. τ_1 preempts τ_2 on core 1, and τ_3 preempts τ_4 on core 2. τ_1 and τ_3 attempt to enter their read phases at time 1050, and τ_1 is granted access while τ_3 is blocked, similar to the case at time 1000. The difference is that at this time τ_2 and τ_4 are preempted, with their intermediate data kept in the core-local memory.
- At time 1056, τ_1 exits its read phase, and τ_3 is granted access and enters its read phase.
- At time 1066, τ_1 enters its write phase since core 2 is not requesting memory access. At time 1071, τ_1 finishes its write phase, and CPU on core 1 yields to τ_2 . τ_2 resumes from preemption and continues its execution phase.
- At time 1072, τ_3 attempts to enter its write phase, and is granted access since τ_2 on core 1 is currently in its execution phase and does not request the shared memory. This is different from time 1022, when τ_3 was blocked by τ_2 since τ_2 was occupying the shared memory.

- At time 1077, τ_3 finishes its write phase, and the CPU on core 2 yields to τ_4 . τ_4 resumes from preemption and continues its execution phase.
- At time 1080, τ_2 exits its execution phase and enters its write phase since the τ_4 on the other core is not requesting the shared memory. τ_2 exits its write phase at time 1090, and core 1 becomes idle.
- At time 1099, τ_4 exits its execution phase and enters its write phase since the other core is not requesting the shared memory.
- At time 1100, a new iteration of τ_1 and τ_3 is released, but both tasks are blocked. τ_3 is blocked by τ_4 , as τ_4 is in its memory phase and cannot be preempted. τ_1 is blocked by τ_4 too, since τ_4 is occupying the shared memory and τ_1 cannot enter its read phase.
- At time 1109, τ_4 exits its write phase, and the scheduler on core 2 selects τ_3 to execute. τ_1 on core 1 has already been waiting, and τ_1 is granted access while τ_3 is blocked. It conforms to the theoretical model under this scenario.
- Scheduling events from time 1109-1136 and time 1150-1177 remains similar to the case in time 1050-1077, with the difference that τ_1 and τ_3 yield to idle tasks instead of τ_2 and τ_4 .

The above is the analysis of the scheduling behavior of the hyperperiod from time 1000 to 1200. Scheduling behavior in other hyperperiods, such as 1200-1400, remains identical. This corresponds to the predictability of the **RTOS** kernel, which aims to eliminate unpredictable behaviors caused by the hardware arbiter on **COTS** platforms, as introduced in section 2.1.1.

6.2 Overhead evaluation

Evaluation of the overheads in the **RTOS** kernel is another important topic in this thesis. In the theoretical model, the overheads are generally considered negligible, although their actual value is not zero. In this section, we introduce how the actual overheads of task switching and inter-core synchronization are evaluated in practice, and how large these overheads are.

6.2.1 Overhead evaluation of task switching

Even if the cost of task switching can be at a minimal value on a simple **RTOS** kernel, it can never be zero as considered in theoretical models, especially when the complexity of the scheduler increases. In this section, we introduce how the overhead of task switching is evaluated on the **RTOS** kernel implemented in this thesis.

To evaluate the task switch time, the hardware cycle counter register CYCCNT is used. By recording the cycle count when entering the scheduler and context switch finishes, the time consumption of task switching can be calculated by the difference. Note that the evaluation is on the hardware level, so the timing consumption of entering and exiting the handler mode, which is done by hardware on Cortex-M platforms, is not evaluated. Nonetheless, this does not obviously affect the evaluation result. Exception enter and return each requires approximately 12 clock cycles under ideal memory conditions on Cortex-M7 platforms [23], which is only a small portion of the entire task switch.

When evaluating the time consumption of task switching, we disable the logging mechanism in section 6.1.1 to have a more accurate result. Logics that only serve for the preemptive mode, such as preemption (section 5.1.3.2) and CPU yield (section 5.1.3.3), are disabled in the fully non-preemptive mode, since tasks are scheduled simply by their priorities then.

Table 6.1 shows the evaluation result of task switching times under different conditions, and figure 6.9 shows the result in the form of a box plot.

Table 6.1: Evaluation result of task switching times (clock cycles)

Mode	Path	Mean	Median	Min	Max
Non-preemptive	–	122	131	102	131
Limited-preemptive	Overall	257	270	194	339
Limited-preemptive	Preemption	199	197	194	209
Limited-preemptive	CPU yield	265	275	198	339

Here we use the same task set as section 6.1.2.3 for evaluation, where preemption thresholds are applied, and tasks are executed on a single core. Time consumption of each concerned task switch is recorded, and evaluations are repeated under different conditions. Task switches between the starting of the **RTOS** kernel and 2000 milliseconds are recorded. Under the fully non-preemptive mode, this time interval contains 122 task switches; under

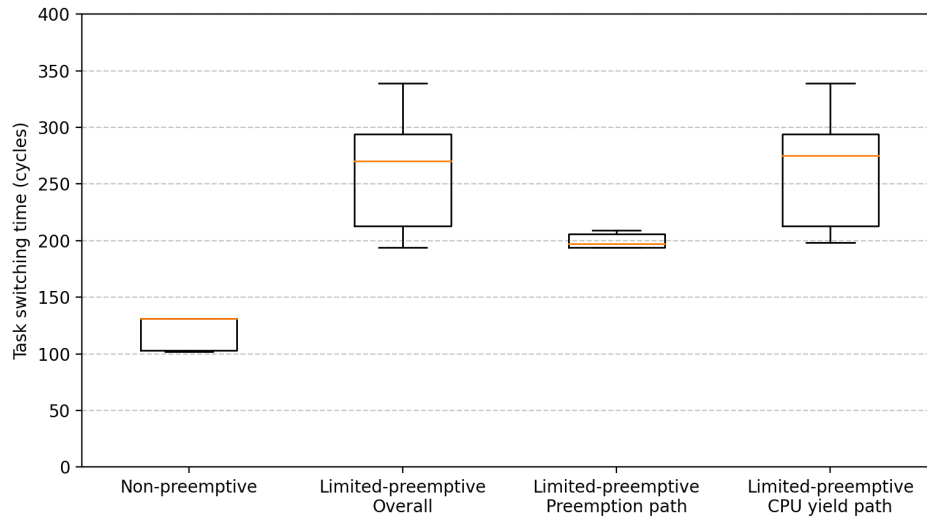


Figure 6.9: Box plot of task switching times

the limited-preemptive mode, 149 task switches occurred, consisting of 17 preemptions and 132 times of CPU yield.

From the statistics, it can be seen that on the tested task set, introducing preemptions to the **RTOS** kernel results in the task switching time increasing by around 135 clock cycles on average, and by around 208 clock cycles in the worst case. On the CPU clock frequency of 300 MHz, which this thesis utilizes, this results in an increase of around 450 nanoseconds on average and 693 nanoseconds in the worst case. With one tick every millisecond, this causes the overhead of the **RTOS** kernel to increase from 0.041% to 0.086% on average, and 0.044% to 0.113% in the worst case.

One more interesting thing that should be noticed is that in the limited-preemptive mode, the overhead of the CPU yield path is significantly greater than the preemption path. This may contradict intuition, but it is reasonable according to the implementation in this thesis.

In section 5.1.3 about scheduler design, we introduced how these two paths in limited-preemptive mode work differently. In the preemption path, priority is the only factor for selecting the next task, as a potential preempting task can never be in its execution phase according to the stack invariant introduced in section 5.1.3.1. Since tasks are denoted by their unique priority and task sets are organised using unsigned integers, finding a task with the highest priority is easily implemented by finding the first non-zero bit of the unsigned integer that represents the task set. This can be done by hardware on Cortex-M7 platforms with a single instruction `CLZ` [24], which is considered to have constant time

complexity.

However, in the CPU yield path, we need to consider preemption thresholds as well, since the next task may be preempted before and is in its execution phase now. Preemption thresholds are not unique and cannot be stored in an unsigned integer, so finding the task with the highest preemption threshold is implemented by traversing through the entire task set in this thesis, which significantly increases time consumption. This can possibly be optimized by maintaining an explicit preemption stack in the **RTOS** kernel. Detailed discussions about this possible optimization will be carried out later in chapter 7.

On the tested task set, CPU yield occurs far more frequently than preemptions in task switching, so the overhead increase on the CPU yield path leads to a significant increase in the overall limited-preemptive mode. Also note that logics such as phase check and memory check only exist in the limited-preemptive mode, and that's why even the preemption path in the limited-preemptive mode shows a slight increase in overhead compared with the fully non-preemptive mode.

6.2.2 Overhead evaluation of inter-core synchronization

The **RTOS** kernel is expected to operate on a dual-core environment, so the coordination between the two cores is also an essential part of the overall overhead evaluation. In section 5.2.2, we introduced the design of the arbiter in this thesis. The logic of arbitration is relatively simple, and the majority of the overhead is caused by covering the time gap between the two cores. Specifically, the synchronization mechanism introduced in section 5.2.1 does not guarantee absolute synchronization between the two cores. A slight timing difference exists and is covered by the waiting windows, which introduces overhead.

To evaluate the timing behaviors of the **RTOS** kernel on dual-core, we trigger a **General-Purpose Input/Output (GPIO)** pulse at the beginning of each systick interrupt on the two cores, and use a logic analyzer to measure the pulses.

Before actually evaluating the timing difference after synchronization, we would like to show that the synchronization mechanism is effective. Figure 6.10 shows systicks on the two cores with the synchronization mechanism disabled, and figure 6.11 shows systicks with synchronization enabled. It can be easily seen that systicks on the two cores do not

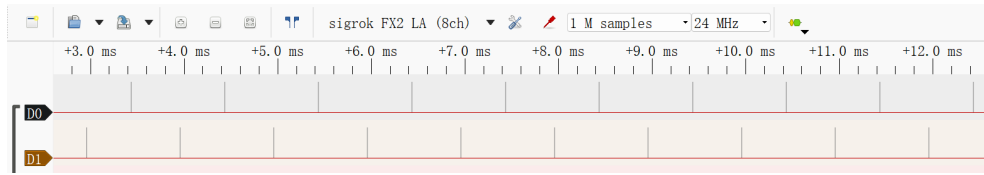


Figure 6.10: Systicks on two cores without synchronization

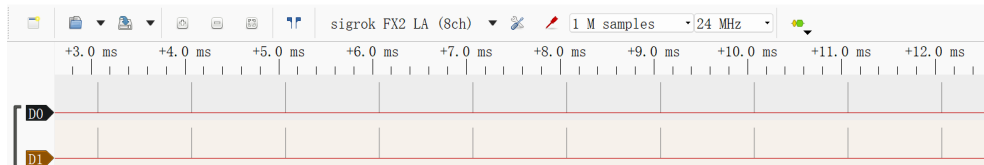


Figure 6.11: Systicks on two cores after synchronization

natively remain synchronized, and this will cause the arbitration logic to fail, which is the same as the example introduced in section 5.2.1. With the synchronization mechanism applied, the systicks on the two cores are relatively well synchronized, with every tick almost triggered at the same time instance.

Even if systicks on the two cores seem to be well synchronized, a slight timing difference still exists. We zoom in on the result in figure 6.11, and measure the time gap between the rising edges of two cores. Figure 6.12 shows the measurement result. It can be seen that after synchronization, each tick on core 2 lags behind that on core 1 by approximately 167 nanoseconds. With the RTOS kernel operating with one tick per millisecond, the coverage of this timing difference introduces approximately 0.017% increase of overhead.

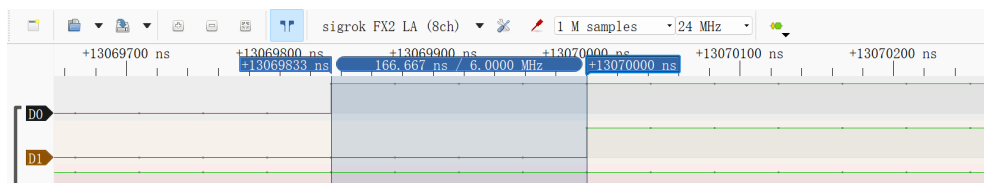


Figure 6.12: Measurement of timing difference

One more thing to be noticed is that the overhead of inter-core synchronization only applies to the core with the leading phase, as the phase-lagging core will find the phase-leading core already registered when entering the arbitration logic, and will directly exit the waiting window. In the case that only one core is requesting access, it needs to wait until the end of the waiting window. This is also an overhead of the arbiter, but it varies with the length

of the waiting window and can be configured.

Chapter 7

Conclusions and future work

This chapter summarizes the main results of the thesis, including the completion status of the expected outcomes, the limitations in practice, possible future work, and an overall reflection.

7.1 Conclusions

In this thesis, we have successfully implemented the expected **RTOS** kernel following the theoretical model and quantitatively evaluated the overheads.

It has been shown in this thesis that the theoretical model of introducing limited preemption to an **RTOS** supporting the three-phase execution model is feasible. The **RTOS** implemented in this thesis supports executing tasks following the three-phase execution model, and preemptions are allowed with the method of **OMPM** and limited by preemption thresholds. We also managed to adapt the kernel to a dual-core environment with the design of arbitration and synchronization. This shows that the theoretical model of the three-phase execution model with the approach of limited preemption can be well implemented in practice, although several limitations exist without contradicting the theoretical model.

Chapter 5 gives the essential steps about the implementation in detail, and the complete source code of the full implementation can be found in appendix A. It is expected that this thesis can provide the basis for the realization of the theoretical model, on which future studies regarding the topic of phased execution models and limited preemption can be carried out.

For the evaluation part of the thesis, we implemented the mechanism for tracing scheduling events, and also gave quantitative evaluations for the overheads of the **RTOS** kernel. As for the most concerning issue, the overhead

of introducing limited preemption, compared to a fully non-preemptive baseline, is precisely evaluated and analyzed. This may possibly suggest correction items for the theoretical model, or provide them with a reference.

Overall, goals proposed in section 1.5 are considered to be met, and the research questions proposed in section 1.2 are answered.

7.2 Limitations

Several limitations exist in the implementation of the **RTOS** kernel. These are due to the trade-offs between complexity, or the difference between theory and practice.

7.2.1 Static assignments of task code

According to the theoretical model, all data that is related to the task should be dynamically loaded, including the task code and task stacks. However, as introduced in section 5.1.2.2, dynamically moving task instructions between different memory locations is not natively supported on Cortex-M platforms. Supporting such features may require modifications to the compiler to generate **PIC**, and significantly exceeds the reasonable workload of this thesis.

7.2.2 Static assignments of task stacks

For the support of dynamic stacks, it should be noted that when a CPU yield happens, a task blocks itself intentionally, which means that the logic of copying intermediate data is executing in the thread mode, and the task stack and the **PSP** register are still in use. If the task stack is moved at this time, the function call from thread mode will have nowhere to return, and errors will occur. This issue can be solved by migrating the logic of intermediate data handling to the handler mode, such as in the PendSV interrupt. However, this will cause the complexity of the **RTOS** kernel to significantly increase, and keeping task stacks statically remaining in the core-local memory does not contradict the theoretical model, as runtime data are the most concerning part.

7.2.3 Task data isolation

In the memory management of this thesis, we restrict tasks to accessing isolated memory slices, which means that memory sharing between tasks is not allowed. This restriction is used to eliminate race conditions during

task executions. Consider the case where a task τ_H with a higher priority preempts a task τ_L with a lower priority, with both tasks attempting to access the same memory slice. τ_H enters its write phase, writing some data to the shared memory. Afterwards, τ_L resumes and enters its write phase, writing some other data to the shared memory and covering the existing data of τ_H . This is a typical case of race conditions. Although mature mechanisms exist for preventing race conditions in shared memory systems, they are not the key topic of this thesis, nor is the theoretical model of introducing limited preemption to phased execution models.

7.2.4 Timing-related limitations

In the theoretical model, task events are considered to occur at discrete synchronized time instances, and task switching is assumed to be instantaneous. In practice, however, task releases and context switches introduce temporal misalignment between the two cores. As a result, arbitration behavior in a real implementation may deviate from the idealized model under continuous execution. Nonetheless, this limitation does not affect the correctness of task scheduling on each core side, nor the support for limited preemption following the method of **OMPM** and preemption thresholds, which are the primary concerns of this thesis.

7.3 Future work

Due to the complexity of the actual environment and the gap between the theoretical model and practice, the **RTOS** kernel implemented in this thesis involves limitations and trade-offs. For the future work that may be carried out on the basis of this thesis, several possible approaches can be considered.

7.3.1 Closer alignment with the theoretical model

In section 7.2, we discussed the limitations of the current implementation of the **RTOS** kernel. Several trade-offs are involved, and they are considered to be improved for a closer alignment with the theoretical model in future work. This includes:

- Supporting loading task code between the shared memory and core-local memory dynamically during memory phases. This may involve compiler-level modifications for generating **PIC**, which supports dynamic load.

- Migrating task stacks along with runtime data during memory phases. This may require handling tasks' intermediate data completely in handler mode and possibly further development of the PendSV interrupt.
- Allowing tasks to access the same memory slices. This requires introducing mechanisms for preventing racing conditions on shared memory and the redesign of the memory management in the **RTOS** kernel.
- Improve timing synchronization between CPU cores and introduce precise timing mechanisms to reduce temporal deviations from the theoretical model.

7.3.2 Maintain an explicit preemption stack

Recall that in the scheduler design introduced in section 5.1.3, we do not explicitly maintain a preemption stack. When a CPU yield occurs, the scheduler needs to traverse through the entire task set to find a task in its execution phase with the highest preemption threshold, which corresponds to the top of the preemption stack.

To reduce the time complexity of finding the top task in the preemption stack, it is feasible to maintain an explicit preemption stack in the form of an array or linked list. With the explicit preemption stack always ready, the time complexity of finding a candidate task can be significantly reduced.

References

- [1] G. Lentaris, K. Maragos, I. Stratakos, L. Papadopoulos, O. Papanikolaou, D. Soudris, M. Lourakis, X. Zabulis, D. Gonzalez-Arjona, and G. Furano, “High-performance embedded computing in space: Evaluation of platforms for vision-based navigation,” *Journal of Aerospace Information Systems*, vol. 15, no. 4, pp. 178–192, 2018. doi: 10.2514/1.I010555. [Online]. Available: <https://doi.org/10.2514/1.I010555>
- [2] D. Dasari, B. Akesson, V. Nélis, M. A. Awan, and S. M. Petters, “Identifying the sources of unpredictability in COTS-based multicore systems,” in *2013 8th IEEE International Symposium on Industrial Embedded Systems (SIES)*, 2013. doi: 10.1109/SIES.2013.6601469 pp. 39–48.
- [3] R. Pellizzoni, E. Betti, S. Bak, G. Yao, J. Criswell, M. Caccamo, and R. Kegley, “A predictable execution model for COTS-based embedded systems,” in *Proceedings of the 17th IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE, 2011, pp. 269–279.
- [4] G. Durrieu, M. Faugère, S. Girbal, D. Gracia Pérez, C. Pagetti, and W. Puffitsch, “Predictable flight management system implementation on a multicore processor,” in *Embedded Real Time Software (ERTS’14)*, Toulouse, France, Feb. 2014. [Online]. Available: <https://hal.science/hal-01121700>
- [5] G. C. Buttazzo, M. Bertogna, and G. Yao, “Limited preemptive scheduling for real-time systems. a survey,” *IEEE Transactions on Industrial Informatics*, vol. 9, no. 1, pp. 3–15, 2013. doi: 10.1109/TII.2012.2188805

- [6] T. Thilakasiri and M. Becker, “Methods to realize preemption in phased execution models,” *ACM Transactions on Embedded Computing Systems*, vol. 22, no. 5s, pp. 153:1–153:25, 2023.
- [7] —, “Work-in-progress: Exploring limited preemption approaches for the phased execution model,” in *Proceedings of the IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2024.
- [8] A. Abel, F. Benz, J. Doerfert, B. Dörr, S. Hahn, F. Hauptenthal, M. Jacobs, A. H. Moin, J. Reineke, B. Schommer, and R. Wilhelm, “Impact of resource sharing on performance and performance prediction: A survey,” in *CONCUR 2013 - Concurrency Theory - 24th International Conference, CONCUR 2013, Buenos Aires, Argentina, August 27-30, 2013. Proceedings*, ser. Lecture Notes in Computer Science, P. R. D’Argenio and H. C. Melgratti, Eds., vol. 8052. Springer, 2013. doi: 10.1007/978-3-642-40184-8_3 pp. 25–43. [Online]. Available: https://doi.org/10.1007/978-3-642-40184-8_3
- [9] C. Maia, L. Nogueira, L. M. Pinho, and D. G. Pérez, “A closer look into the AER model,” in *2016 IEEE 21st International Conference on Emerging Technologies and Factory Automation (ETFA)*, 2016. doi: 10.1109/ETFA.2016.7733567 pp. 1–8.
- [10] J. Arora, C. Maia, S. A. Rashid, G. Nelissen, and E. Tovar, “Schedulability analysis for 3-phase tasks with partitioned fixed-priority scheduling,” *Journal of Systems Architecture*, vol. 131, p. 102706, 2022. doi: 10.1016/j.sysarc.2022.102706
- [11] Y. Wang and M. Saksena, “Scheduling fixed-priority tasks with preemption threshold,” in *Proceedings Sixth International Conference on Real-Time Computing Systems and Applications. RTCSA’99 (Cat. No.PR00306)*, 1999. doi: 10.1109/RTCSA.1999.811269 pp. 328–335.
- [12] S. Baruah, “The limited-preemption uniprocessor scheduling of sporadic task systems,” in *Proceedings of the 17th Euromicro Conference on Real-Time Systems (ECRTS)*. IEEE, 2005, pp. 137–146.
- [13] R. Tabish, R. Mancuso, S. Wasly, S. S. Phatak, R. Pellizzoni, and M. Caccamo, “A reliable and predictable scratchpad-centric os for multi-core embedded systems,” in *2017 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2017. doi: 10.1109/RTAS.2017.1 pp. 377–388.

- [14] C. Pagetti, J. Forget, H. Falk, D. Oehlert, and A. Luppold, “Automated generation of time-predictable executables on multicore,” in *Proceedings of the 26th International Conference on Real-Time Networks and Systems*, ser. RTNS '18. New York, NY, USA: Association for Computing Machinery, 2018. doi: 10.1145/3273905.3273907. ISBN 9781450364638 p. 104–113. [Online]. Available: <https://doi.org/10.1145/3273905.3273907>
- [15] G. Yao, R. Pellizzoni, S. Bak, H. Yun, and M. Caccamo, “Global real-time memory-centric scheduling for multicore systems,” *IEEE Transactions on Computers*, vol. 65, no. 9, pp. 2739–2751, 2016. doi: 10.1109/TC.2015.2500572
- [16] W.-H. Huang, J.-J. Chen, and J. Reineke, “Mirror: Symmetric timing analysis for real-time tasks on multicore platforms with shared resources,” in *2016 53rd ACM/EDAC/IEEE Design Automation Conference (DAC)*, 2016. doi: 10.1145/2897937.2898046 pp. 1–6.
- [17] J. Arora, S. A. Rashid, C. Maia, and E. Tovar, “Analyzing fixed task priority based memory centric scheduler for the 3-phase task model,” in *2022 IEEE 28th International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA)*, 2022. doi: 10.1109/RTCSA55878.2022.00012 pp. 51–60.
- [18] *RM0483: SR5E1x 32-bit Arm Cortex-M7 architecture microcontroller for electrical vehicle applications*, STMicroelectronics, October 2024, revision 5.0. [Online]. Available: https://www.st.com/resource/en/reference_manual/rm0483-sr5e1x-32bit-arm-cortexm7-architecture-microcontroller-for-electrical-vehicle-applications-stmicroelectronics.pdf
- [19] D. Comer and T. V. Fossum, *Operating System Design, Vol. I: The XINU Approach (PC Edition)*. Prentice Hall, 1988. ISBN 0-13-638180-4
- [20] Arm Ltd., “Procedure call standard for the Arm architecture (AAPCS),” <https://github.com/ARM-software/abi-aa>, 2020, ARM IHI 0042J.
- [21] C. L. Liu and J. W. Layland, “Scheduling algorithms for multiprogramming in a hard-real-time environment,” *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973. doi: 10.1145/321738.321743
- [22] K. Wang, “Embedded real-time operating systems,” in *Embedded and Real-Time Operating Systems*. Springer, 2023, pp. 429–503.

- [23] *ARM Cortex-M7 Processor Technical Reference Manual*, r1p2 ed., Arm Limited, 2018, accessed: 2026-05-24. [Online]. Available: <https://developer.arm.com/documentation/ddi0489/latest>
- [24] *ARMv7-M Architecture Reference Manual*, ARM Limited, 2010. [Online]. Available: <https://developer.arm.com/documentation/ddi0403/latest>

Appendix A

Complete source code repository

The complete source code, including the full implementation of the **RTOS** kernel, as well as the visualization tools, can be found at <https://github.com/Reeenyi/PredictOS>.

It is expected that all work introduced in this thesis can be reproduced with the source code in an identical or equivalent development environment.

